



Multi-objective distributed flow shop scheduling with inter-factory transportation and variable production speed

Lydia Hilarius , Udo Buscher & Tristan Becker

To cite this article: Lydia Hilarius , Udo Buscher & Tristan Becker (10 Aug 2026): Multi-objective distributed flow shop scheduling with inter-factory transportation and variable production speed, International Journal of Production Research, DOI: [10.1080/00207543.2026.2713120](https://doi.org/10.1080/00207543.2026.2713120)

To link to this article: <https://doi.org/10.1080/00207543.2026.2713120>



© 2026 The Author(s). Published by Informa UK Limited, trading as Taylor & Francis Group.



Published online: 10 Aug 2026.



[Submit your article to this journal](#)



Article views: 533



[View related articles](#)



[View Crossmark data](#)

Multi-objective distributed flow shop scheduling with inter-factory transportation and variable production speed

Lydia Hilarius^a, Udo Buscher^b and Tristan Becker^a

^aFaculty of Business and Economics, Germany, Junior Professorship in Business Management, esp. Management Science, TU Dresden, Dresden, Germany; ^bFaculty of Business and Economics, Germany, Chair of Business Administration, esp. Industrial Management, TU Dresden, Dresden, Germany

ABSTRACT

In a globally integrated economy, production is frequently distributed across multiple factories or production lines, a challenge addressed by the Distributed Flow Shop Scheduling Problem. Our study introduces two key extensions: (1) the option to increase production speed at additional cost, and (2) inter-factory transportation of semi-finished products, incurring transportation costs and delay, for the benefit of higher factory utilisation. While most existing research focuses on makespan, we consider total tardiness to generate customer-oriented schedules. We introduce a multi-objective approach to minimise total tardiness and the costs of inter-factory transportation and production speed variation, yielding Pareto-optimal solutions. To efficiently solve this problem, we propose an iterated greedy algorithm tailored to the problem characteristics. Computational experiments demonstrate that our method outperforms mathematical and constraint programming approaches. Our findings quantify the individual and combined value of transportation and production speed adjustments and indicate that their joint use can yield synergistic improvements.

ARTICLE HISTORY

Received 11 February 2026
Accepted 24 July 2026

KEYWORDS

Scheduling; distributed flow shop; multi-objective optimisation; inter-factory transportation; variable production speed; total tardiness

2020 MATHEMATICS SUBJECT CLASSIFICATIONS

90B35; 90C29; 90C59

1. Introduction

In the networked economy, optimisation of production, logistics, and industrial processes is essential to maintain competitiveness and ensure customer satisfaction. Scheduling plays a central role in this context, as it generates plans that maximise the utilisation of scarce resources, particularly in manufacturing (Ruiz, Pan, and Naderi 2019).

Flow Shop Scheduling Problems (FSP) involve processing a set of jobs on multiple machines following the same route. Their distributed extension, the Distributed Flow Shop Scheduling Problem (DFSP), reflects the growing interconnectedness of modern industry by considering multiple factories or production lines with identical machine equipment and has been first described by Naderi and Ruiz (2010). The DFSP is NP-hard and has been studied extensively, yielding both exact and heuristic solution approaches (Naderi, Ruiz, and Roshanaei 2023; Perez-Gonzalez and Framinan 2024; Ruiz, Pan, and Naderi 2019).

However, existing DFSP research is limited in two important respects. First, it rarely considers inter-factory transportation, i.e. the transfer of semi-finished products between factories, which can improve resource utilisation. This approach is already being adopted by

the industry, particularly in the semiconductor sector. While multi-site supply chain planning acknowledges this aspect (Moon, Kim, and Hur 2002; Thoney et al. 2002), it was only recently formalised as the DFSP with inter-factory transportation (DFSPT) by Becker, Neufeld, and Buscher (2025). Second, research on variable production speeds has mainly focussed on reducing energy consumption and CO₂ emissions (Schulz, Buscher, and Shen 2020). In contrast, increasing production speeds can be deliberately used to shorten processing times, mitigate bottlenecks, and improve due-date adherence, particularly in tightly coupled production stages. This, however, comes at the cost of higher energy consumption, machine wear, and waste.

In this paper, we integrate both aspects into a unified framework: the Multi-Objective Distributed Flow Shop Scheduling Problem with Inter-Factory Transportation and Variable Production Speed (MO-DFSP-TS). We address three objectives simultaneously. The first objective is to minimise total tardiness, a customer-oriented measure that captures the increasing importance of due date adherence in distributed manufacturing (He et al. 2025; Khare and Agrawal 2021). Despite its relevance, total tardiness has rarely been studied in the DFSP context (Perez-Gonzalez and Framinan 2024), where

research has largely emphasised makespan. The second and third objectives minimise the costs of inter-factory transportation and increased production speeds, thereby enabling a structured analysis of trade-offs between tardiness reduction and incurred expenses. Unlike makespan, this tardiness-based approach creates a different economic dynamic: costly interventions like speed-ups and transports are only justified if they actively prevent a job from missing its deadline. This perspective provides actionable insights on when to implement flexibility mechanisms to ensure customer satisfaction. Given the multi-objective nature of the problem, our analysis focuses on Pareto-optimal solutions rather than isolated optima.

Methodologically, we propose mathematical programming (MP) and constraint programming (CP) formulations. Additionally, we introduce a novel multi-objective iterated greedy metaheuristic, building upon previous work on FSP (Ruiz and Stützle 2007), DFSP (Ruiz, Pan, and Naderi 2019), DFSPT (Becker, Neufeld, and Buscher 2025), and multi-objective scheduling with variable production speeds (Schulz, Schönheit, and Neufeld 2022). A computational study evaluates the performance of the proposed algorithms and quantifies the value of inter-factory transportation and variable production speeds in reducing total tardiness.

Section 1.1 provides a review of the current state of research on this topic. Section 1.2 describes the contribution of this paper, and Section 1.3 outlines its organisation.

1.1. Literature review

Exhaustive literature reviews on DFSP can be found in works such as Perez-Gonzalez and Framinan (2024) and Lohmer and Lasch (2021). We therefore focus this section on the literature most relevant to our setting. First, we provide a general overview of DFSP research, before addressing objective functions in more detail. Next, we examine the characteristics of MO-DFSP-TS, reviewing recent studies on transportation in FSP and on variable production speeds. To provide a structured comparison, Table 1 summarises the most relevant literature, categorising studies by the scheduling problem, the specific flexibilities (transportation and speed), the objectives, the exact formulations, and the applied solution methods.

Distributed Flow Shop Scheduling Problems. Naderi and Ruiz (2010) formalise the Distributed Permutation Flow Shop Scheduling Problem (DPFSP) by generalising the Permutation Flow Shop Scheduling Problem (PFSP)

to manufacturing in multiple sites rather than in a single factory. In their review, Perez-Gonzalez and Framinan (2024) distinguish three clusters of extensive research on DPFSP: (1) Classic DPFSP with a makespan objective, (2) Constrained DPFSP, and (3) Multi-objective DPFSP. Recent research on clusters (1) and (2) often extends the DPFSP to more accurately reflect real-world production settings, typically with the objective of minimising the makespan. Considered characteristics include subdividing jobs into families in group scheduling problems (Han et al. 2025; He et al. 2024; B. Zhang 2025) and modelling heterogeneous factories with different capabilities (Feng et al. 2024; Xiong, Chen, et al. 2025; Xiong, Jing, et al. 2025). Further studies investigate sequence-dependent setup times (Song, Lin, and Chen 2025; C. Zhang et al. 2025; Zhong et al. 2026) as well as the integration of assembly operations and their coordination with completion times (Z.-Q. Zhang et al. 2025; Zhao et al. 2023). Moreover, the problem is exacerbated by blocking constraints (Ribas, Companys, and Tort-Martorell 2019; S. Wang et al. 2024; Y. Wang et al. 2025) and the consideration of dynamics and uncertainties (Fathollahi-Fard, Woodward, and Akhrif 2024; H. Li et al. 2025). We discuss the multi-objective DPFSP (cluster (3)) in the paragraph on multi-objective approaches.

Solution algorithms for DPFSP comprise exact and heuristic solution approaches. Exact formulations include mixed-integer linear programming (MILP) based on sequence or positions (Naderi and Ruiz 2010). Recently, Naderi, Ruiz, and Roshanaei (2023) demonstrated that Constraint Programming (CP) formulations outperform MILP formulations for a range of scheduling problems including the DPFSP. However, exact methods are typically only suitable for small instances, due to the inherent complexity of the problem. Therefore, many studies focus on heuristic approaches. Naderi and Ruiz (2010) present approximate solution methods by decomposing the DPFSP into two decisions: (1) Allocating jobs to factories and (2) determining the job sequence. The authors propose a neighbourhood-based solution algorithm, building upon existing heuristic approaches and dispatching rules (Ruiz and Stützle 2007). Recent research on DFSP follows these ideas. To construct a solution, most studies adapt the NEH heuristic and apply problem-specific dispatch rules. Subsequently, various heuristic and metaheuristic approaches improve initial solutions. Common methods include population-based approaches, such as artificial bee colony algorithms (Han et al. 2025; C. Zhang et al. 2025). Many studies adopt iterated greedy-based strategies due to their simplicity, transferability to various problems, and performance (Feng et al. 2024; He

Table 1. Review of relevant literature on Distributed Flow Shop Scheduling Problems with inter-factory transportation and production speed variation.

Author	Probl.			Flexib.			Objectives				Exact		Algorithm						
	DFSP	HFSP	FSP	TP-I	TP-O	SP	C _{max}	FT	T	EC	MO	MP	CP	Decomp.	EA	IG	Neighb.	RL/NN	SI
Becker, Neufeld, and Buscher (2025)	●	○	○	●	○	○	●	○	○	○	○	●	●	○	○	●	○	○	○
Cai, Yang, and Liu (2018)	●	○	○	○	●	○	●	○	●	○	●	●	○	○	●	○	○	○	○
Chen, Cheng, and Chou (2020)	○	●	○	○	○	●	●	○	○	●	●	●	○	○	●	○	○	○	○
Ding et al. (2021)	○	○	●	○	○	●	○	○	●	●	●	●	○	○	○	○	○	○	●
Fu et al. (2022)	●	○	○	○	●	○	●	○	○	○	○	●	○	○	●	○	○	○	●
Fathollahi-Fard, Woodward, and Akhrif (2024)	●	○	○	○	○	●	●	○	○	○	○	●	○	○	○	○	●	○	○
Feng et al. (2024)	●	○	○	○	○	○	○	○	●	○	○	●	○	○	○	●	○	○	○
Gao et al. (2024)	●	○	○	○	●	○	●	○	○	○	●	●	○	○	●	○	○	●	○
Hou et al. (2023)	●	○	○	○	●	○	○	○	●	○	●	●	○	○	○	○	○	○	●
Han et al. (2025)	●	○	○	○	○	○	○	○	●	○	○	●	○	○	○	○	○	○	○
He et al. (2024)	●	○	○	○	○	○	○	●	○	○	○	●	○	○	○	●	○	○	○
Khare and Agrawal (2021)	●	○	○	○	○	○	○	○	●	○	○	●	○	○	○	●	○	○	○
H. Li et al. (2025)	●	○	○	○	○	○	●	○	○	○	○	●	○	○	○	○	○	○	○
W. Li et al. (2022)	●	○	○	○	○	○	●	○	○	○	○	●	○	○	○	○	○	○	○
H. Li, Li, and Gao (2021)	●	○	○	○	○	○	●	○	○	○	○	●	○	○	○	○	○	○	○
Q. Li et al. (2021)	●	○	○	○	○	○	○	○	○	○	○	●	○	○	○	○	○	○	○
Naderi and Ruiz (2010)	●	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○
Naderi, Ruiz, and Roshanaei (2023)	●	●	●	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○
Ribas, Companys, and Tort-Martorell (2019)	●	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○
Ruiz and Stützel (2007)	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○
Schulz, Schönheit, and Neufeld (2022)	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○
Schulz, Buscher, and Shen (2020)	○	●	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○
Song, Lin, and Chen (2025)	●	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○
Y. Wang et al. (2025)	●	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○
S. Wang et al. (2024)	●	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○
Wu and Che (2020)	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○
Xiong, Chen, et al. (2025)	●	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○
Xiong, Jing, et al. (2025)	●	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○
Xiong, Shi, et al. (2025)	●	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○
Xu et al. (2025)	●	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○
Yu et al. (2024)	●	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○
B. Zhang (2025)	●	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○
C. Zhang et al. (2025)	●	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○
Z.-Q. Zhang et al. (2025)	●	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○
Zhao et al. (2024)	●	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○
Zhao et al. (2023)	●	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○
Zhong et al. (2026)	●	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○
This work	●	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○

Scheduling Problem (Probl.): **DFSP:** Distributed Flow Shop; **HFSP:** Hybrid Flow Shop; **FSP:** Other type of Flow Shop.

Flexibility (Flexib.): **TP-I:** Inter-factory transportation; **TP-O:** Other transportation; **SP:** Production speed variation.

Objectives: **C_{max}:** Makespan related; **FT:** Flow time related; **T:** Tardiness related; **EC:** Energy consumption related; **MO:** Multi-objective.

Exact models (Exact): **MP:** Mathematical programming; **CP:** Constraint programming.

Algorithm: **Decomp.:** Decomposition techniques/mathheuristics; **EA:** Evolutionary algorithms; **IG:** Iterated greedy; **Neighb.:** Neighbourhood-based; **RL/NN:** Reinforcement learning/neural networks; **SI:** Swarm intelligence.

et al. 2024; Y. Wang et al. 2025; Zhong et al. 2026). These strategies typically consist of an initialisation, an iterative loop of destruction, construction, and improvement, as well as an acceptance and stop criterion. Researchers frequently expand on this approach by adding problem-specific search and improvement mechanisms. While DPFSP has been studied extensively, the literature focuses makespan minimisation, which limits its ability to capture more complex and practically relevant decision trade-offs.

Total Tardiness in FSP. Khare and Agrawal (2021) and Ribas, Companys, and Tort-Martorell (2019) present the first comprehensive studies of the DFSP with the objective of minimising total tardiness. While Ribas, Companys, and Tort-Martorell (2019) propose an iterated greedy algorithm for parallel blocking FSP that extends to DFSP, Khare and Agrawal (2021) explicitly examine the distributed version and introduce two metaheuristics to solve the problem: a hybrid discrete Harris Hawks optimisation algorithm, a population-based metaheuristic, and an enhanced variant of an iterated greedy algorithm. Han et al. (2025); Xiong, Chen, et al. (2025) and Feng et al. (2024) propose metaheuristic solution approaches for extended variants of the problem, including artificial bee colony, adaptive evolutionary, and iterated greedy algorithms. Hou et al. (2023); Y. Wang et al. (2025) include a delivery problem minimising total weighted earliness and tardiness. To the best of our knowledge, the research of Hou et al. (2023) is the only one that adapts a multi-objective approach with tardiness for DFSP. Overall, total tardiness has received comparatively little attention in DFSP.

Multi-Objective Approaches in DFSP. The vast majority of multi-objective DFSP studies focus on energy efficiency or emissions reduction. Therefore, the multi-objective approaches examined here resemble those reviewed in relation to variable production speeds. The solution approaches encompass greedy- and neighbourhood-based heuristics (Schulz, Schönheit, and Neufeld 2022; Zhong et al. 2026), population-based heuristics (Hou et al. 2023; C. Zhang et al. 2025), hybrid and learning-based strategies (Yu et al. 2024; Z.-Q. Zhang et al. 2025; Zhao et al. 2023), and exact and hybrid-exact approaches (Xiong, Shi, et al. 2025; B. Zhang 2025). Most of these contributions aim at approximating the Pareto front of the considered objectives. Alternative scalarization strategies are less common and mainly include weighted-sum formulations (Xiong, Shi, et al. 2025) or ε -constraint-type approaches, where one objective is

optimised subject to explicit bounds on the remaining ones (C. Zhang et al. 2025). In contrast to single-objective problems, the definition, selection, and analysis of performance metrics, particularly for Pareto-based approaches, constitute a significant part of multi-objective research (Datta and Figueira 2012). Our review of the objectives in recent research on DFSP reveals that, although a substantial amount of research employs multi-objective approaches, few studies address tardiness objectives.

Transportation in FSP. Transportation occurs at various points in the supply chain, including the supply of raw materials to production facilities, transportation processes within the production process, and delivery to customers after production. The literature models these stages in several ways. H. Li, Li, and Gao (2021); Xiong, Shi, et al. (2025) and Cai, Yang, and Liu (2018) incorporate specific transportation times and efforts required to supply jobs to factories, resulting in decision-dependent release dates. W. Li et al. (2022) indirectly address transportation endeavours before and after production, such as the implementation of a loading and unloading robot with limited capacity. Research on post-completion transportation encompasses delivery endeavours for aggregated batches (Q. Li et al. 2021) and the emissions caused per job and factory (Schulz, Buscher, and Shen 2020). Fu et al. (2022) and Hou et al. (2023) incorporate a downstream capacitated vehicle routing problem.

Our MO-DFSP-TS study focuses on transportation within the production process. Until recently, the literature has only addressed the additional transportation efforts required between machines, assembly stages, and production cells (Gao et al. 2024; Xu et al. 2025; B. Zhang 2025). Becker, Neufeld, and Buscher (2025) present the DFSPT, which introduces the transportation of semi-finished products between factories to enhance capacity utilisation. In contrast to previous studies, the incorporation of inter-factory transportation can change both factory assignments and job sequences. This approach deviates from the widely adopted permutation assumption. The authors propose two formulations: a sequence-based MILP with additional decision variables for factory allocation and transportation, and a CP formulation with interval variables for jobs and sequence variables for jobs and factories. Furthermore, they propose an iterated greedy-based approach with a speed-up procedure for evaluating insertions, which outperforms the MILP and CP formulations. In their paper, Becker, Neufeld, and Buscher (2025) demonstrate that the makespan can be significantly reduced through inter-factory transportation. However, they do not account for customer-oriented objectives such as

total tardiness and neglect transportation costs. As a result, the literature lacks a comprehensive analysis of the trade-offs between the costs and benefits of inter-factory transportation.

Variable Production Speeds in FSP. There is an extensive research on selecting the production speed of each machine from a discrete set of speed levels. Recent literature primarily aims at reducing energy consumption by lowering production speeds, yet energy savings diminish with further speed reductions. Most studies analyze the trade-offs between makespan or total flow time and energy-related objectives (Chen, Cheng, and Chou 2020; Schulz, Schönheit, and Neufeld 2022; Wu and Che 2020; Yu et al. 2024; Z.-Q. Zhang et al. 2025; Zhao et al. 2023, 2024). The total tardiness objective is only addressed alongside an energy-related objective for flexible flow shops (Ding et al. 2021) and hybrid flow shops (Schulz, Buscher, and Shen 2020).

To efficiently solve these complex multi-objective scheduling problems, most contributions propose heuristics and metaheuristics. Similar to previously described multi-objective approaches, they include iterated greedy- and neighbourhood-based approaches (Schulz, Schönheit, and Neufeld 2022; Wu and Che 2020; Zhong et al. 2026), population-based evolutionary algorithms (Chen, Cheng, and Chou 2020; Ding et al. 2021; Zhao et al. 2024), as well as as hybrid and learning-based strategies (Yu et al. 2024; Z.-Q. Zhang et al. 2025; Zhao et al. 2023). These approaches typically construct solutions at the highest speed level and then gradually decrease the speed levels of non-critical operations. Recent studies on DFSP incorporate variable production speeds as an energy-saving strategy and analyze the trade-off with makespan. However, the impact of variable production speeds on total tardiness in DFSP, including inter-factory transportation, remains unexplored.

1.2. Contribution

The contributions of this study are as follows.

- We introduce a novel multi-objective extension of the DFSP that jointly integrates inter-factory transportation and production speed variation. In addition, we expand beyond makespan-based evaluations by incorporating total tardiness, thereby aligning the optimisation with due date adherence and customer-orientation. By explicitly modelling transportation and speed variation costs alongside total tardiness, we are able to systematically analyze the trade-offs between service quality and operational flexibility.
- We develop exact and heuristic solution approaches for the proposed multi-objective problem. We introduce MP and CP formulations as exact methods and propose a tailored multi-objective iterated greedy metaheuristic. The metaheuristic is specifically designed to address the structural characteristics of distributed flow shops with transportation and speed variation and enables the efficient generation of high-quality Pareto sets.
- We conduct an extensive computational study to assess the performance of the proposed solution approaches and to analyze the value of the two flexibility options. The results demonstrate the effectiveness and scalability of the metaheuristic compared to exact methods. Beyond algorithmic performance, our analysis provides insight into the trade-offs between inter-factory transportation, production speed variation, and total tardiness, and highlights complementary and synergistic benefits.

1.3. Organisation of this paper

The remainder of this paper is organised as follows. Section 2 introduces the problem and presents mathematical programming and constraint programming formulations, along with the proposed multi-objective exact solution approach. Section 3 describes the multi-objective iterated greedy metaheuristic. Section 4 reports the computational study, evaluating the performance of the exact approaches and the proposed metaheuristic, and analyzes the value of inter-factory transportation and production speed variation for reducing total tardiness. Finally, Section 5 concludes the paper and outlines directions for future research.

2. Problem description and mathematical formulations

In this section, we describe the MO-DFSP-TS in detail and provide an illustrative example. Furthermore, we present our MP and CP formulations, as well as our exact multi-objective solution approach.

2.1. Problem description

The MO-DFSP-TS allocates jobs with specific due dates to a set of distributed factories. During production, jobs can be transported between factories, and the production speed of each operation may vary. The MO-DFSP-TS aims to minimise three objectives: (1) the total tardiness, defined as the sum of all job delays, (2) the transportation cost between factories, and (3) the cost of production speed variation.

We consider a set of factories $k \in K$, each of which has an identical set of $i \in I$ machines. Each job $j \in J$ must be processed on each machine in the same sequence across any of the distributed factories. Furthermore, each job has a specific due date d_j by which it should be completed. We assume preemptions are not allowed and there is no blocking. Each machine can only process one job at a time and each job can be processed only on one machine at a time.

The following characteristics expand upon the conventional DPFSP:

- *Inter-factory transportation*: After each operation on machine i each job j can be transported from the current factory k to another factory k' . Job j then continues to be processed on machine $i + 1$ in factory k' after transportation time $t_{kk'}$.
- *Production speed variation*: Each operation of a job j on a machine i is assigned a speed level from a discrete set $v \in V$, which determines the respective processing time p_{ijv} .

We address three objectives:

- (1) Minimise the total tardiness, denoted by $\mathcal{TT}$, which is defined as the sum of the nonnegative deviation between the completion times and the due dates for all jobs.
- (2) Minimise the cost of inter-factory transportation, denoted by $\mathcal{TP}$, which is the sum of the distance-weighted transports.
- (3) Minimise the cost of production speed variations, denoted by $\mathcal{SP}$, which depend on the speed level and processing time of each operation.

Example 2.1: We provide an example to illustrate the impact of inter-factory transportation and production speed variation on total tardiness. To allow for intuitive interpretation, this example focuses on total tardiness, only reporting transport and speed-up costs incurred by minimising $\mathcal{TT}$. Table 2 provides the exemplary data for two factories, three machines, and five jobs. Figure 1 illustrates the optimal schedules for different combinations of the flexibility options of the DFSP.

First, Figure 1(a) shows the schedule that minimises total tardiness without inter-factory transportation and speed variation. The resulting objective value is $\mathcal{TT} = 106$, primarily due to delays of Jobs 2 and 4. Second, we introduce the option of inter-factory transportation without production speed variation. In this example, we assume homogeneous distances of $t_{kk'} = 5$ between factories, meaning that each transport takes five time units. As demonstrated in Figure 1(b), two transports of Jobs 1 and 3, both between Machines 2 and 3, reduce total tardiness by nine units. This is achieved by reducing the delays of Jobs 2 and 4, resulting in a total tardiness of $\mathcal{TT} = 97$. Third, we allow for a 10% increase in production speed without including inter-factory transportation. As shown in Figure 1(c), increasing the production speed of 12 out of 15 operations substantially improves the schedule and reduces the delay of Job 4, resulting in a total tardiness of $\mathcal{TT} = 63.3$.

Fourth, we combine inter-factory transportation and production speed variation. Figure 1(d) shows the resulting schedule. Although the tardiness of Jobs 1 and 5 increases compared to the schedule in Figure 1(a), the significant improvement for Jobs 2 and 4 results in an overall reduction in total tardiness by 61.1 units to $\mathcal{TT} = 44.9$. Interestingly, the reduction in total tardiness in this combined case exceeds the sum of the individual improvements from inter-factory transportation and production speed variation. This example demonstrates the advantages of integrating inter-factory transport and production speed variation.

2.2. MP formulation

In this section, we present a MP formulation for the MO-DFSP-TS. It builds upon the general DFSP formulation proposed by Naderi, Ruiz, and Roshanaei (2023), as well as the extensions introduced by Schulz, Schönheit, and Neufeld (2022) for variable production speeds and by Becker, Neufeld, and Buscher (2025) for inter-factory transportation. The notation of our MP and CP formulations is summarised in Table 3. To model the MO-DFSP-TS, we represent the objective functions $\mathcal{TT}$, $\mathcal{TP}$, and $\mathcal{SP}$ by introducing additional decision variables. Continuous variables C_{ij} denote the completion time of job j on machine i . Binary variables $x_{ijj'}$ reflect

Table 2. Processing times and due dates for an exemplary DFSP with 2 factories, 3 machines, and 5 jobs.

M_i / J_j	J_1	J_2	J_3	J_4	J_5
M_1	26	38	27	88	95
M_2	55	54	63	23	45
M_3	86	43	43	40	37
Due Date	191	154	138	154	206

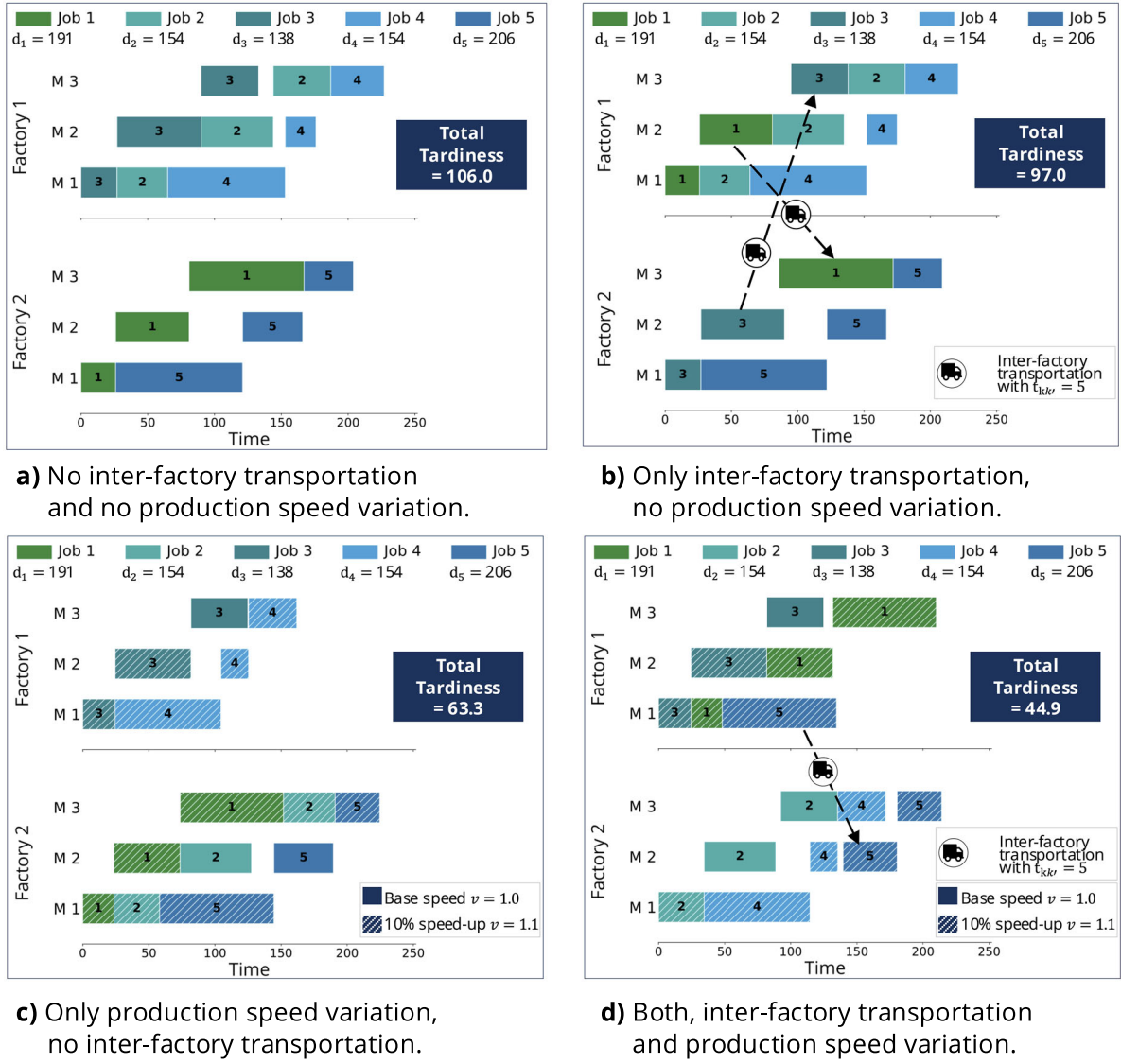


Figure 1. Illustrative example for minimum total tardiness with and without inter-factory transportation and production speed variation.

the job precedence by indicating whether job j' precedes job j on machine i . Binary variables y_{kijv} reflect whether factory k processes job j on machine i at speed level v . Finally, binary variables $z_{kk'ij}$ model inter-factory transportation and take the value 1 if job j is transported from factory k to factory k' after being processed on machine i .

$$s_v = (v_v)^{v_v} - 1 \quad \forall v \in V \quad (1)$$

$$p_{ijv}^s = \frac{p_{ij}}{v_v} \quad \forall i \in I, \forall j \in J, \forall v \in V \quad (2)$$

$$TT = \sum_{j \in J} T_j \quad (3)$$

$$TP = \sum_{k \in K} \sum_{k' \in K} \sum_{i \in I} \sum_{j \in J} (z_{kk'ij} \cdot t_{kk'}) \quad (4)$$

$$SP = \sum_{k \in K} \sum_{i \in I} \sum_{j \in J} \sum_{v \in V} y_{kijv} \cdot s_v \cdot p_{ij} \quad (5)$$

$$T_j \geq (C_{mj} - d_j) \quad \forall j \in J \quad (6)$$

$$C_{1j} \geq y_{k1jv} \cdot p_{1jv}^s \quad \forall k \in K, \forall j \in J, \forall v \in V \quad (7)$$

$$C_{ij} \geq C_{(i-1)j} + y_{kijv} \cdot p_{ijv}^s + \sum_{k' \in K} (z_{kk'(i-1)j} \cdot t_{kk'})$$

$$\forall k \in K, \forall i \in I \setminus \{1\}, \forall j \in J, \forall v \in V \quad (8)$$

$$\sum_{k \in K} \sum_{v \in V} y_{kijv} = 1 \quad \forall i \in I, \forall j \in J \quad (9)$$

$$\sum_{v \in V} y_{k(i+1)jv} \geq \sum_{v \in V} y_{kijv} - \sum_{k' \in K} (z_{kk'ij} + z_{k'kij})$$

$$\forall k \in K, \forall i \in \{1, \dots, m-1\}, \forall j \in J \quad (10)$$

Table 3. Notation for MP and CP formulations.

Index set	Description
$J = \{1, \dots, n\}$	Set of jobs
$I = \{1, \dots, m\}$	Set of machines
$K = \{1, \dots, f\}$	Set of factories
$V = \{1, \dots, q\}$	Set of speed levels
Parameters	Description
p_{ij}	Processing time of job j on machine i
p_{ijv}^s	Adjusted processing time of job j on machine i at speed level v
d_j	Due date of job j
$t_{kk'}$	Cost and time of transportation between factory k and k'
s_v	Cost of production speed variation to speed level v
v_v	Speed factor of production speed variation at speed level v
M	Sufficiently large constant
Decision variables (MP)	Description
$\mathcal{TT}$	Total tardiness
$\mathcal{TP}$	Cost of inter-factory transportation
$\mathcal{SP}$	Cost of production speed variation
C_{ij}	Completion time of job j on machine i
$x_{ijj'}$	1 if job j' precedes job j on machine i ; 0 otherwise
y_{kijv}	1 if factory k processes job j on machine i at speed level v ; 0 otherwise
$z_{kk'ij}$	1 if job j is transported from factory k to factory k' after machine i ; 0 otherwise
Decision variables (CP)	Description
R_{ij}	Interval variable for the processing interval of job j on machine i
W_{kijv}	Optional interval variable for the processing interval job j on machine i in factory k at speed level v
MS_{ki}	Sequence variable for jobs on machine i of factory k
FS_j	Sequence variable for the factory sequence of job j

$$\begin{aligned}
C_{ij'} &\geq C_{ij} + y_{kij'v} \cdot p_{ij'v}^s \\
&\quad - M \cdot (2 + x_{ijj'} - \sum_{v \in V} (y_{kijv} + y_{kij'v})) \\
&\quad \forall k \in K, \forall i \in I, \forall j \in J, \forall j' \in J | j > j', \forall v \in V \quad (11)
\end{aligned}$$

$$\begin{aligned}
C_{ij} &\geq C_{ij'} + y_{kijv} \cdot p_{ijv}^s \\
&\quad - M \cdot \left(3 - x_{ijj'} - \sum_{v \in V} (y_{kijv} + y_{kij'v}) \right) \\
&\quad \forall k \in K, \forall i \in I, \forall j \in J, \forall j' \in J | j > j', \forall v \in V \quad (12)
\end{aligned}$$

$$C_{ij}, T_j, \mathcal{TT}, \mathcal{TP}, \mathcal{SP} \geq 0 \quad \forall i \in I, \forall j \in J \quad (13)$$

$$\begin{aligned}
x_{ijj'}, y_{kijv}, z_{kk'ij} &\in \{0, 1\} \\
&\quad \forall k, k' \in K, \forall i \in I, \forall j \in J, \forall v \in V \quad (14)
\end{aligned}$$

We define the marginal cost of production speed variation in Equations 1, adopting an exponential relation between production speed and cost increase from Schulz, Schönheit, and Neufeld (2022). Specifically, we model the cost as an exponential function of the speed factor v . At

a speed factor of 1, the nominal processing times remain unchanged, and no additional cost is incurred. Furthermore, we determine the adjusted processing time p_{ijv}^s by dividing the nominal processing time p_{ij} by the speed factor v_v as shown in Equations 2.

Tardiness T_j is defined as the nonnegative difference between a job's maximum completion time C_{mj} and its due date d_j (Constraints 6 and 13). The objective function 3 calculates total tardiness $\mathcal{TT}$ as the sum of these differences across all jobs. The objective function 4 determines the transportation cost between factories $\mathcal{TP}$ based on the inter-factory distance ($t_{kk'}$) and the corresponding binary variables ($z_{kk'ij}$). Finally, the objective function 5 captures the cost of production speed variation $\mathcal{SP}$, which depends on the marginal speed cost s_v , the nominal processing time of the operation p_{ij} , and binary variables y_{kijv} .

Furthermore, we initialise the completion time C_{1j} for the first operation of each job in Constraints 7, which is defined by its corresponding processing time. The completion time for subsequent operations C_{ij} is captured in Constraints 8, depending on the completion time of the previous operation $C_{(i-1)j}$, the processing time of the operation p_{ijv}^s , and the potential transportation time $t_{kk'}$. Next, Constraints 9 ensure that each job is processed on all machines and that each operation has exactly one speed level and factory. We ensure the continuity of factory assignments and transportation in Constraints 10 by requiring that consecutive operations of a job must either remain in the same factory or involve transportation. Constraints 11 and 12 define the job precedence and prevent job overlaps. The first part of Constraints 11 applies only if j' is a successor of j on machine i in the same factory k . It defines the completion time of job j' based on the previous job j and its processing time $p_{ij'v}^s$. Conversely, the first part of Constraints 12 applies only if j' is a predecessor of j on machine i in the same factory k , and it defines its completion time accordingly. Finally, Constraints 13 and 14 specify the domains of the decision variables.

2.3. CP formulation

We build our CP formulation upon the models proposed by Naderi, Ruiz, and Roshanaei (2023) and Becker, Neufeld, and Buscher (2025), and adopt the syntax of the IBM ILOG CP Optimiser. The objective functions 15–17 correspond to those of the MP formulation but have been adapted to CP syntax.

$$\mathcal{TT}^{CP} = \sum_{j \in J} \max(R_{jm} - d_j, 0) \quad (15)$$

$$\mathcal{TP}^{CP} = \sum_{k \in K} \sum_{k' \in K} \sum_{i \in I \setminus \{m\}} \sum_{j \in J} \sum_{v \in V} \text{presenceOf}(W_{kijv}) \cdot \text{presenceOf}(W_{k'(i+1)jv}) \cdot t_{kk'} \quad (16)$$

$$\mathcal{SP}^{CP} = \sum_{k \in K} \sum_{i \in I} \sum_{j \in J} \sum_{v \in V} \text{presenceOf}(W_{kijv}) \cdot p_{ij} \cdot s_v \quad (17)$$

$$R_{ij} = \text{IntervalVar}() \quad \forall i \in I, \forall j \in J \quad (18)$$

$$W_{kijv} = \text{IntervalVar}\left(\frac{p_{ij}}{v_v}\right) \quad \forall k \in K, \forall i \in I, \forall j \in J, \forall v \in V \quad (19)$$

$$\text{Alternative}(R_{ij}, W_{kijv} : k \in K, v \in V) \quad \forall i \in I, \forall j \in J \quad (20)$$

$$\text{MS}_{ki} = \text{SequenceVar}(W_{kijv} : j \in J, v \in V) \quad \forall k \in K, \forall i \in I \quad (21)$$

$$\text{NoOverlap}(\text{MS}_{ki}) \quad \forall k \in K, \forall i \in I \quad (22)$$

$$\text{EndBeforeStart}(R_{(i-1)j}, R_{ij}) \quad \forall i \in I \setminus \{1\}, \forall j \in J \quad (23)$$

$$\text{FS}_j = \text{SequenceVar}(W_{kijv} : k \in K, i \in I, v \in V) \quad \forall j \in J \quad (24)$$

$$\text{NoOverlap}(\text{FS}_j, t_{kk'}) \quad \forall j \in J, \forall k \in K, \forall k' \in K \quad (25)$$

In Constraints 18, we introduce interval variables R_{ij} , which represent the start and end times of job j on machine i . Furthermore, Constraints 19 define optional interval variables W_{kijv} , which are present if job j is processed at speed level v and in factory k on machine i . The size of the interval accounts the adjusted processing time under speed variation, $p_{ijv}^s = p_{ij}/v_v$. Constraints 20 enforce that each operation (i, j) selects exactly one factory k and speed level v , determining the size of interval R_{ij} . Constraints 21 define sequence variables MS_{ki} that determine the processing order of jobs for each factory k and machine i . Furthermore, Constraints 22 and 23 prevent overlaps in the machine sequences MS_{ki} and subsequent operations of the same job. Constraints 24 define sequence variable FS_j , which models the factory sequence of each job j across machines. Finally, Constraints 25 model inter-factory transportation by including the transportation time $t_{kk'}$ as an offset between subsequent intervals whenever a job changes factories within FS_j . Here, $t_{kk'}$ is a transition matrix that specifies transport times in case a job changes between factories k and k' while $t_{kk} = 0 \forall k \in K$.

2.4. Multi-objective approach

We aim to minimise three objectives: total tardiness $\mathcal{TT}$, transportation cost $\mathcal{TP}$, and speed cost $\mathcal{SP}$. However,

the units of the objectives differ and there is no straightforward way to assign meaningful weights a priori. This argument also applies to the cost objectives $\mathcal{TP}$ and $\mathcal{SP}$, for which no generally applicable weighting can be assumed across different manufacturing environments. Therefore, we aim to identify the Pareto front, i.e. the set of Pareto-efficient solutions. A solution is Pareto-efficient if no objective can be improved without deteriorating at least one other. We use two different approaches to approximate the Pareto fronts of the MP and CP formulations. For the MP formulation, we apply the augmented ε -constraint method (AUGMECON) introduced by Mavrotas (2009), which efficiently covers the entire Pareto front, including non-convex regions. For the CP formulation, we use a weighted sum approach to leverage the superior convergence and search efficiency of CP solvers when optimising a combined objective function. The extreme points of the Pareto fronts represent the solutions for the respective single-objective sub-problems. Beyond the multi-objective context, both exact formulations can be simplified to single-objective problems by focussing on an individual objective.

In the following, we outline the ε -constraint method, which converts a multi-objective problem into a series of single-objective optimisation problems. This approach optimises the first objective while incorporating the other objectives as model constraints that are gradually relaxed. The AUGMECON method improves upon the standard ε -constraint approach by ensuring that only efficient solutions are generated, thereby reducing computational effort.

$$\min \quad \frac{\mathcal{TT}}{\text{span}^{\mathcal{TT}}} + \delta \cdot \left(\frac{\text{slack}^{\mathcal{TP}}}{\text{span}^{\mathcal{TP}}} + \frac{\text{slack}^{\mathcal{SP}}}{\text{span}^{\mathcal{SP}}} \right) \quad (26)$$

$$\mathcal{TP} - \text{slack}^{\mathcal{TP}} = \varepsilon^{\mathcal{TP}} \quad (27)$$

$$\mathcal{SP} - \text{slack}^{\mathcal{SP}} = \varepsilon^{\mathcal{SP}} \quad (28)$$

$$\text{slack}^{\mathcal{TP}}, \text{slack}^{\mathcal{SP}} \geq 0 \quad (29)$$

Objective function 26 minimises total tardiness $\mathcal{TT}$, incorporating objectives $\mathcal{TP}$ and $\mathcal{SP}$ through slack variables, multiplied by a small number δ to consider only Pareto-efficient solutions. Variables $\text{slack}^{\mathcal{TP}}$ and $\text{slack}^{\mathcal{SP}}$, defined in Constraints 27–29, allow the ε -Constraints to be slightly violated, but penalise these violations proportionally to the scale of each objective. Parameters $\text{span}^{\mathcal{TT}}$, $\text{span}^{\mathcal{TP}}$, and $\text{span}^{\mathcal{SP}}$ denote the ranges between the maximum and minimum values of each objective. These values are determined through lexicographic optimisation and are used to normalise the objective values. We vary the values $\varepsilon^{\mathcal{TP}}$ and $\varepsilon^{\mathcal{SP}}$ iteratively across the spans of the objective functions $\mathcal{TP}$ and $\mathcal{SP}$ according to a selected number of sampling points.

In addition, we employ the weighted-sum method in the objective function 30 to solve the CP model. Using a discretization parameter $steps$, we generate a set of weight vectors, which yields distinct objective-weight combinations: $\Omega(steps) = \{(\omega^{TT}, \omega^{TP}, \omega^{SP}) :$

$$\omega^o = \frac{s^o}{steps}, s^o \in \{0, \dots, steps\},$$

$$forallo \in \{TT, TP, SP\},$$

$$\sum_{o \in \{TT, TP, SP\}} s^o = steps\}$$

$$\min \quad \omega^{TT} \cdot TT + \omega^{TP} \cdot TP + \omega^{SP} \cdot SP \quad (30)$$

3. Multi-objective iterated greedy metaheuristic

In this section, we introduce our multi-objective iterated greedy (MO-IG) approach for efficiently solving the MO-DFSP-TS. The complexity of the MO-DFSP-TS extends that of the DFSP due to two additional degrees of freedom: (1) the possibility of inter-factory transportation, which eliminates the permutation constraint, and (2) the production speed variation, which requires an additional decision regarding the speed level for each operation. To address the inherent complexity of the MO-DFSP-TS, we select an iterated greedy-based approach. Iterated greedy metaheuristics are widely recognised as state-of-the-art for DFSP and many studies have demonstrated their performance for FSP (Perez-Gonzalez and Framinan 2024; Ruiz and Stützle 2007). Such approaches have proven effective for DFSP variants, including inter-factory transportation (Becker, Neufeld, and Buscher 2025), variable production speeds (Schulz, Schönheit, and Neufeld 2022), and tardiness objectives (Feng et al. 2024; Khare and Agrawal 2021; Ribas, Companys, and Tort-Martorell 2019). Building on this strong foundation in single-objective settings, iterated greedy-based frameworks have also been widely adopted for multi-objective scheduling (Ciavotta, Minella, and Ruiz 2013; Schulz, Schönheit, and Neufeld 2022; Xiong, Shi, et al. 2025; Zhong et al. 2026).

Section 3.1 describes our solution representation and schedule construction, Section 3.2 discusses the strategy of our proposed algorithmic framework, and Section 3.3 outlines the algorithmic framework and details the key components.

3.1. Solution representation and schedule construction

In contrast to most existing DFSP studies, which assume a permutation-based solution representation, our approach employs a complete (non-permutation) representation. Existing DPFSP heuristics typically describe a

solution through the job-to-factory assignment and per-factory permutations that define the processing order. However, this representation is not valid for the MO-DFSP-TS, since inter-factory transportation allows operations of the same job to be processed in different factories, thereby violating the permutation property. Furthermore, the solution representation needs to account for the speed level $v \in V$ of each operation. We adopt an integrated representation that specifies the assignment of each operation of job j at speed level v on machine i in factory k in an ordered set $\Psi_{ki} = \{(j_1, v_1), \dots, (j_n, v_n)\}$.

To evaluate this representation, we deterministically construct the corresponding schedule by calculating the exact completion times. The ordered set Ψ_{ki} inherently provides the complete factory and speed assignments. For the evaluation of a schedule, let k_{ij} denote the assigned factory and v_{ij} the selected speed level for job j on machine i . Furthermore, let α_{ij} denote the job immediately preceding job j on machine i in factory k_{ij} , as defined by the sequence in $\Psi_{k_{ij}i}$. If j is the first job on the machine, $\alpha_{ij} = 0$ with a completion time of zero.

The completion time C_{ij} of job j on machine i is determined recursively. Let $p_{ijv_{ij}}^s$ be the adjusted processing time of job j on machine i at the specifically assigned speed level v_{ij} . We define C_{ij}^{tp} as the completion time including the subsequent transportation delay $t_{k_{ij}k_{(i+1)j}}$ to the factory of the next machine. If $k_{ij} = k_{(i+1)j}$, the transportation time is zero.

$$C_{1j} = C_{1\alpha_{1j}} + p_{1jv_{1j}}^s \quad \forall j \in J \quad (31)$$

$$C_{1j}^{tp} = C_{1j} + t_{k_{1j}k_{2j}} \quad \forall j \in J \quad (32)$$

$$C_{ij} = \max(C_{i\alpha_{ij}}, C_{(i-1)j}^{tp}) + p_{ijv_{ij}}^s \quad \forall i \in I \setminus \{1\}, \forall j \in J \quad (33)$$

$$C_{ij}^{tp} = C_{ij} + t_{k_{ij}k_{(i+1)j}} \quad \forall i \in I \setminus \{1, m\}, \forall j \in J \quad (34)$$

Upon calculating the final completion times C_{mj} , the tardiness of each job is obtained as $T_j = \max(0, C_{mj} - d_j)$. To calculate the final objective values for total tardiness (TT), inter-factory transportation costs (TP), and speed variation costs (SP), we explicitly map the assignments from Ψ_{ki} to the binary variables y_{kijv} and $z_{kk'ij}$ as follows:

$$y_{kijv} = 1 \quad \text{if } k_{ij} = k \quad \text{and} \quad v_{ij} = v \quad (0 \text{ otherwise}) \quad (35)$$

$$z_{kk'ij} = 1 \quad \text{if } k_{ij} = k \quad \text{and} \quad k_{(i+1)j} = k' \neq k \quad (0 \text{ otherwise}) \quad (36)$$

Using this mapping, the objective function values are exactly calculated according to our model definitions (Equations 3–5).

3.2. Metaheuristic design principles

The characteristics of the MO-DFSP-TS—including inter-factory transportation, variable speeds, tardiness, and multiple objectives—motivate the following metaheuristic design principles:

- *Operation-based search procedure:* We employ operation-based search mechanisms across all search operators of our multi-objective iterated greedy metaheuristic, which is detailed in Section 3.3. Consequently, the evaluation of (re)insertions into (partial) schedules is based on operations (i, j) , rather than on complete jobs j , as is typical in iterated greedy algorithms for flow shop scheduling problems. Becker, Neufeld, and Buscher (2025) have shown that this approach outperforms job-based search procedures with inter-factory transportation. Furthermore, the operation-based approach is well suited to the tardiness objective, as completion times and tardiness are influenced by both sequencing and speed decisions and accumulate toward later operations of each job.
- *Two-stage speed selection:* The dependence of processing times on the selected speed level complicates the search, as sequencing and speed decisions interact. We implement a two-stage approach suggested by Ding et al. (2021) to determine production speed levels. First, we assign jobs to machines and factories at maximum speed level. Second, we determine the optimal speed level, mitigating bottlenecks while allowing non-critical operations to be executed more efficiently. Sections 3.3.1 and 3.3.3 further detail our construction and local search procedures that entail this approach.
- *Bound-strengthening techniques:* To improve the lower bound on the total tardiness and transportation costs when evaluating partial schedules, we employ two bound-strengthening mechanisms considering (1) the effects of speed variation and (2) the effects of inter-factory transportation: (1) When inserting an operation (i, j) , we tighten the lower bound on the completion times by accounting for the processing times of unassigned operations at maximum speed, improving the accuracy of tardiness evaluation. (2) Whenever a job changes factories, at least one transport is required. This applies, when operation $(i - 1, j)$ is processed in factory k , operation $(i + 1, j)$ is processed in another factory $k' \neq k$ and operation (i, j) is unassigned in the (partial) schedule. We improve the lower bounds on TT and TP by adding the inter-factory distance $t_{kk'}$ to the lower bounds of both the successor's completion time and the transportation cost, whenever the partial solution implies such a transition. Preliminary experiments indicated that these mechanisms improve the

lower bounds of total tardiness and transportation cost, leading to a more focussed search and faster convergence, with moderate impact on the final solution quality.

- *Randomised tie-breaking mechanism:* To avoid a systematic bias toward any of the three objectives, we resolve ties between candidate solutions through a stochastic tie-breaking rule. If multiple solutions yield the same scalarised objective value for a given weight vector ω^o , we select a candidate uniformly at random. Preliminary experiments indicated that more elaborate tie-breaking rules proposed in the literature (e.g. based on idle times or earliness (Fernandez-Viagas and Framinan 2015)) disproportionately favoured total tardiness. Randomised tie-breaking ensures a more balanced exploration of the objective space. Algorithms 2–4 showcase the specific application of our tie-breaking mechanism.

3.3. Algorithm framework and procedures

Our algorithm, summarised in Algorithm 1, follows the general structure of iterated greedy frameworks, consisting of an initialisation phase, a destruction-repair mechanism to introduce diversification, a local search to explore the local neighbourhood, and an acceptance criterion. In contrast to single-objective variants, our multi-objective extension maintains a set of Pareto-efficient solutions rather than a single incumbent. To evaluate insertions within the algorithmic procedures, we utilise weight sets. In our MO-IG, we determine the best insertion by comparing all factory, position, and speed options and taking the combination with the smallest weighted objective value.

The algorithm begins by generating an initial Pareto set, *ParetoSet*. We further detail the construction procedure in Section 3.3.1. Subsequently, the algorithm iteratively refines this set until reaching the time limit. Each iteration is based on a randomly generated weight vector $\omega^{rnd,o}$. We propose two different strategies for selecting a solution s from *ParetoSet* to improve in the respective iteration. (1) *SelectRnd*, which chooses a solution at random from *ParetoSet*, and (2) *SelectBest*, which limits the random selection to a subset of the highest-quality solutions according to $\omega^{rnd,o}$. Next, the algorithm creates a copy s' of the selected solution s and successively applies two operators. (1) *DESTRUCTREPAIR*, defined in Algorithm 3 and detailed in Section 3.3.2, and (2) *LOCALSEARCH*, defined in Algorithm 4 and detailed in Section 3.3.3.

After each operator, *DESTRUCTREPAIR* and *LOCALSEARCH*, s' undergoes a Pareto-efficiency check as an

Algorithm 1 Multi-Objective Iterated Greedy Meta-heuristic (MO-IG)

```

1: procedure MULTI-OBJECTIVE ITERATED GREEDY
2:   for  $\omega^{init,o}$  in initial weight set do
3:      $s \leftarrow$  Initialization using CONSTRUCTION( $\omega^{init,o}$ )
4:      $s \leftarrow$  Improve  $s$  using LOCALSEARCH( $s, \omega^{init,o}$ )
5:     if  $s$  is not dominated by ParetoSet then
6:       Add  $s$  to ParetoSet
7:       Remove solutions dominated by  $s$  from ParetoSet
8:     end if
9:   end for
10:  while time limit not reached do
11:    for random weight  $\omega^{rnd,o}$  do
12:       $s' \leftarrow$  Copy solution  $s$  selected from ParetoSet based on
        SelectRnd or SelectBest
13:       $s' \leftarrow$  DESTRUCTREPAIR( $s', NumDestr, \omega^{rnd,o}$ )
14:      if  $s'$  is not dominated by any solution in ParetoSet then
15:        Add  $s'$  to ParetoSet
16:        Remove solutions dominated by  $s'$  from ParetoSet
17:      end if
18:       $s' \leftarrow$  LOCALSEARCH( $s', \omega^{rnd,o}$ )
19:      if  $s'$  is not dominated by any solution in ParetoSet then
20:        Add  $s'$  to ParetoSet
21:        Remove solutions dominated by  $s'$  from ParetoSet
22:      end if
23:    end for
24:  end while
25:  return ParetoSet
26: end procedure

```

acceptance criterion. If s' is Pareto-efficient, it is added to *ParetoSet* (Algorithm 1, lines 14–17 and 18–22). Accordingly, we eliminate dominated solutions that are part of *ParetoSet*. This archiving process ensures the overall quality of the approximated Pareto front. Crucially, by evaluating newly generated solutions strictly against the global *ParetoSet*, we prevent the algorithm from accepting progressively worse solutions during the search. Furthermore, the use of varying weight vectors $\omega^{rnd,o}$ prevents stagnation in local regions. In single-objective applications, the framework can be simplified by replacing the *ParetoSet* with a single incumbent and substituting the Pareto-efficiency check with a standard acceptance criterion, such as Simulated Annealing. The next sections describe the initialisation, the destruction–repair mechanism, and the local search phase in detail.

3.3.1. Initialisation procedure

We present the CONSTRUCTION procedure in detail in Algorithm 2, while Figure 2 illustrates its main components, including the operation-based two-stage insertion.

We generate an initial set of schedules s by repeatedly executing the construction and local search procedures for a uniformly distributed set of weight vectors $\omega^{init,o}$ over the three objectives $o \in \{TT, TP, SP\}$. The initialisation procedure is based on the NEH heuristic (Nawaz, Enscore, and Ham 1983). It starts by sorting all jobs according to the Earliest Due Date (EDD) rule into an ordered set J^{EDD} (Algorithm 2, line 2). Subsequently, it

inserts the operations (i, j) of each job into the partial solution s , in accordance with the sequence of J^{EDD} and employing a two-stage speed selection strategy. First, it determines the best insertion at the maximum speed level $\max(V)$ (Algorithm 2, lines 4–7), and second, it selects the best speed level v for the given position and factory of operation (i, j) (Algorithm 2, lines 8–10). Figure 2 illustrates the construction procedure for an example with two machines and two factories.

Algorithm 2 Construction Procedure

```

1: procedure CONSTRUCTION( $\omega^{init,o}$ )
2:    $J^{EDD} \leftarrow$  Sort jobs  $j \in J$  by EDD
3:   for each  $j$  in  $J^{EDD}$  do
4:     for each  $i$  in  $I$  do
5:       Determine best insertion for  $(i, j)$  at  $\max(V)$  considering
        TieBREAK
6:        $s \leftarrow$  Insert operation  $(i, j)$  at best position and factory at
         $\max(V)$ 
7:     end for
8:     for each  $i$  in  $I$  do
9:        $s \leftarrow$  Select best speed  $v$  for  $(i, j)$  at given position and
        factory considering TieBREAK
10:    end for
11:  end for
12:  return  $s$ 
13: end procedure

```

In the first stage, we assign operation $(1, j)$ to position 1 in factory 1 and operation $(2, j)$ to position 0 in factory 2. In the second stage, we attempt to decrease the speed level, keeping the maximum speed level $v = 2$ for operation $(1, j)$ and reducing the speed of operation $(2, j)$ to $v = 1$. As described in Algorithm 1, line 4, after adding all jobs from J^{EDD} , each solution s undergoes the local search procedure (Algorithm 4, Section 3.3.3). Finally, if s is not dominated by any solution in *ParetoSet*, it is added to *ParetoSet* and any solutions dominated by s are eliminated (see Algorithm 1, lines 5–8).

3.3.2. Destruction-repair procedure

We introduce a Destruction–Repair operator (DESTRUCTREPAIR) to enhance the diversity and exploration of the search process. This operator builds upon the approach proposed by Becker, Neufeld, and Buscher (2025). In contrast to job-based destruction-repair mechanisms in iterated greedy algorithms, our approach operates at the level of individual operations (i, j) . We begin by randomly selecting a subset of operations and removing it from the current (partial) schedule (Algorithm 3, lines 3–7), where hyperparameter *NumDestr* prescribes the number of operations to remove. Afterwards, we reinsert each operation according to the best insertion option at factory k and speed level v (Algorithm 3, lines 8–11).

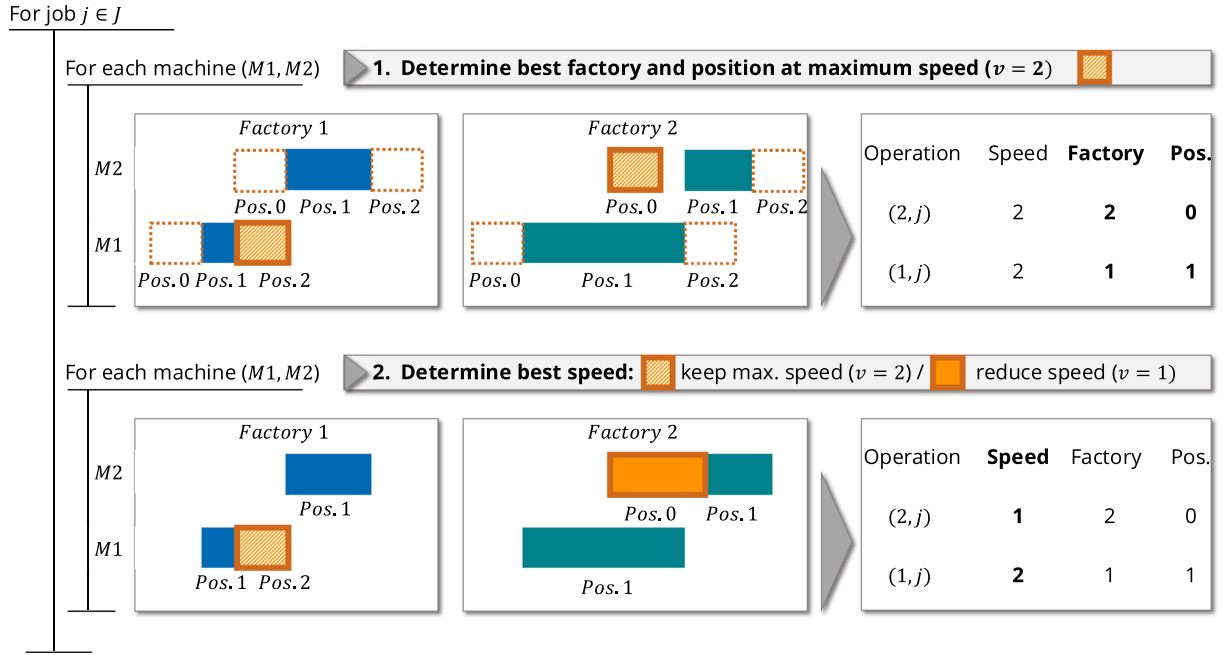


Figure 2. Illustration of the operation-based construction procedure (see Algorithm 2) including the two-stage speed selection for two machines and two factories as part of our MOIG algorithm.

Algorithm 3 Destruction and Repair Operator

```

1: procedure DESTRUCTREPAIR( $s', NumDestr, \omega^{rnd,o}$ )
2:    $D \leftarrow \emptyset$ 
3:   for  $idx = 1 \rightarrow NumDestr$  do
4:      $(i, j) \leftarrow$  Select random operation  $(i, j)$  from current (partial)
       solution  $s'$ 
5:     Delete  $(i, j)$  from  $s'$ 
6:      $D \leftarrow D \cup \{(i, j)\}$ 
7:   end for
8:   for each  $(i, j)$  in  $D$  do
9:     Determine best insertion for operation  $(i, j)$  considering
       TIEBREAK
10:     $s' \leftarrow$  Insert operation  $(i, j)$  at best position, factory, and speed
       level
11:   end for
12:   return  $s'$ 
13: end procedure

```

3.3.3. Local search procedure

Our local search procedure (Algorithm 4) tries to improve the current solution s for a given weight vector ω^o (either $\omega^{init,o}$ or $\omega^{rnd,o}$) by iteratively exploring its neighbourhood. We include it both during initialisation (Algorithm 1, line 4) and in each iteration (Algorithm 1, line 18) of the MO-IG. To diversify the search, the local search processes jobs according to a random permutation of jobs J^{rnd} (Algorithm 4, line 6). For each job $j \in J^{rnd}$, it examines all operations $i \in I$ using a copy s' of the current solution s (Algorithm 4, lines 6–7). LOCALSEARCH employs a two-stage operation-based procedure. First, it selects the best position and factory for all operations (i, j) of the current job at maximum speed ($\max(V)$) (Algorithm 4, lines 12–15). Second, it iterates each

operation of the considered job again to determine the best speed levels v at the previously determined position and factory (Algorithm 4, lines 16–18). After this two-stage determination of the best positions, factories, and speed levels for all operations of a job, the algorithm computes the objective value $ObjVal$ (Algorithm 4, lines 19–24) and accepts solution s' if it improves upon the initial solution s ; otherwise, it is rejected. The search repeats over J^{rnd} until it finds no further improvement.

4. Computational study

We conducted a computational study to assess both the performance of the proposed solution approaches and the value of inter-factory transportation and production speed variations for reducing total tardiness. First, we compare the exact formulations with the MO-IG metaheuristic to evaluate their computational efficiency and solution quality. Second, we analyze the trade-offs between total tardiness, inter-factory transportation, and production speed variation.

Section 4.1 introduces our experimental setting, Section 4.2 defines the performance metrics employed in our computational study and Section 4.3 presents the results of the hyperparameter experiment. Subsequently, Sections 4.4 and 4.5 analyze the computational performance of the solution approaches and the impact of inter-factory transportation and production speed variation in distributed flow shops.

Algorithm 4 Local Search Procedure

```

1: procedure LOCALSEARCH( $s, \omega^o$ )
2:    $improve = true$ 
3:   while  $improve$  do
4:      $ObjVal(s) \leftarrow$  Determine objective value from current schedule
        $s$  at  $\omega^o$ 
5:      $improve = false$ 
6:      $J^{nd} \leftarrow$  Random permutation of jobs
7:     for  $j$  in  $J^{nd}$  do
8:        $s' \leftarrow$  Create a copy of  $s$ 
9:       for  $i$  in  $I$  do
10:        Remove  $(i, j)$  from  $s'$ 
11:      end for
12:      for  $i$  in  $I$  do
13:        Determine best insertion for  $(i, j)$  at  $\max(V)$  considering TieBREAK
14:       $s' \leftarrow$  Insert operation  $(i, j)$  at best position and factory at  $\max(V)$ 
15:      end for
16:      for  $i$  in  $I$  do
17:         $s' \leftarrow$  Select best speed  $v$  for  $(i, j)$  at given position and factory considering TieBREAK
18:      end for
19:       $ObjVal(s') \leftarrow$  Determine objective value of  $s'$  at  $\omega^o$ 
20:      if  $ObjVal(s') < ObjVal(s)$  then
21:         $s \leftarrow s'$ 
22:         $improve = true$ 
23:      break
24:    end if
25:  end for
26:  end while
27:  return  $s$ 
28: end procedure

```

4.1. Experimental setting

We implemented the exact formulations based on the mathematical models given in Sections 2.2–2.4 in Gurobi 12.0 for the MP formulation and IBM ILOG CP Optimiser 22.1 for the CP formulation. We implemented the proposed metaheuristic in C++.

We conducted all experiments on the TU Dresden High Performance Computing cluster on a virtual machine running Ubuntu 22.04 with 3 vCPUs mapped to an AMD EPYC 7513 (2.6 GHz) physical core, equipped with 256 GB of RAM. We parallelised the main loop of the metaheuristic (Algorithm 1, Lines 10–24) to distribute iterations across the available CPU cores. To ensure the efficient utilisation of computing resources, we limited the runtime of hyperparameter experiments for the MO-IG to 600 s. The runtime for all final runs of MP, CP, and MO-IG was 1800 s. To ensure coverage of a wide range of the Pareto front within this designated time frame, we introduced local time limits for the individual sampling points of the MP and CP formulations. Specifically, we set the number of steps to 10 and limited each sampling point to a maximum runtime of 300 s. This prevents the solver from exhausting the total time limit on a single, computationally expensive point, thereby facilitating the exploration of different regions of the Pareto front.

Table 4. Speed factor v_v for different numbers of speed levels ($|V| = \{3; 5\}$) in the computational study.

$ V / v \in V$	1	2	3	4	5
5	1.0	1.05	1.15	1.3	1.5
3	1.0	1.1	1.25		

We used benchmark instances based on the instance generator of Taillard (1993) and the extension of Khare and Agrawal (2021) by due dates. For all instances, we sampled job processing times from a uniform random distribution between 1 and 99. We rounded the adjusted processing times after speed variation p_{ijv}^s , as shown in Equation 2, to an integer to ensure applicability in constraint programming and comparability between solution approaches. We calculated the due date of each job based on the total processing time, the number of factories in the instance, and a random variable *rand* uniformly distributed between 0 and 1, following Khare and Agrawal (2021) (see Equation 37). All test instances are published in the Online Appendix.

Furthermore, we set a homogeneous inter-factory distance with $t_{kk'} \in \{5, 10, 20, 50\} \forall k \in K, \forall k' \in K$ for $k \neq k'$ and $t_{kk} = 0$. For the hyperparameter experiments, we utilised a subset of $t_{kk'} \in \{5, 10, 20\} \forall k \in K, \forall k' \in K$ for $k \neq k'$ and $t_{kk} = 0$, while the final evaluations (Sections 4.4–4.5) include a distance of 50 to analyze the system's behaviour under high-distance constraints. These ranges are consistent with the distance scales identified by Becker, Neufeld, and Buscher (2025) as yielding significant transportation effects in the DFSP. Given our processing times, the selected values represent fast transfers, typical for industrial clusters like semiconductor manufacturing or integrated steel mills with multiple production lines or factories located in close proximity to one another.

For the hyperparameter tuning and the study of computational performance, we considered two possible numbers of speed levels: $|V| = \{3, 5\}$, with specific speed factors listed in Table 4. To better evaluate the impact of speed variation, we narrowed the scope to two speed levels ($v = 1.0$ and 1.1) in Section 4.5.2.

$$d_j = \sum_{i \in I} p_{ij} \cdot (1 + rand \cdot (1/f) \cdot 0.5) \quad \forall j \in J \quad (37)$$

We distinguished between a set of small and a set of large instances, which include the following numbers of jobs, machines, and factories:

- Small: $n = \{5, 8, 10\}, m = \{3, 5\}, f = \{2, 4\}$
- Large: $n = \{10, 20, 50\}, m = \{10, 20\}, f = \{2, 4, 6\}$

For each combination of n , m , and f , we generated three distinct test instances, $id = \{1, 2, 3\}$ using different

random seeds. One of these instances $id = \{1\}$ was designated for the hyperparameter experiments described in Section 4.3, while the other instances $id = \{2, 3\}$ were employed for the final experiments evaluated in Sections 4.4–4.5. These instances were combined with both speed levels and the three distance levels. The total number of test instances is $n \times m \times f \times |t_{kk'}| \times |V| \times |id|$, resulting in 72 small and 108 large instances for the hyperparameter experiments and 192 small and 288 large instances for the final experiments.

4.2. Performance metrics

We applied performance metrics to assess the quality of Pareto fronts and to facilitate the comparison of different solution approaches. For each approach $h \in H$, we denote the set of non-dominated solutions it produces as the Pareto front PF_h . Each solution $x \in PF_h$ is evaluated with respect to the three objectives $TT(x)$, $TP(x)$, and $SP(x)$. We employed three complementary metrics that have been widely adopted in recent multi-objective studies (Chang, Jia, and Ren 2025; Cheng et al. 2025; Ciavotta, Minella, and Ruiz 2013; Ghorbani Saber and Ranjbar 2022; He et al. 2025; Schulz, Schönheit, and Neufeld 2022): (1) Hypervolume indicator (HV), (2) Inverted generational distance (IGD), and (3) C metric. The HV indicator focuses on the dominated objective space and thus emphasises both convergence and distribution from a global perspective. The IGD measures the average proximity of the approximated Pareto front to a reference front, reflecting convergence and the geometric spread along the reference set. Finally, the C metric quantifies the percentage of solutions in one front that are weakly dominated by another, providing a pairwise comparison of Pareto fronts (Datta and Figueira 2012; Neufeld, Schulz, and Buscher 2023).

To ensure consistent scaling, all objective values are normalised by dividing each objective by its global maximum across all fronts, resulting in values between 0 and 1. Let $\mathcal{P} = \bigcup_{h \in H} PF_h$ denote the set of all Pareto-optimal solutions obtained by all approaches.

$$\begin{aligned}\overline{TT}(x) &= \frac{TT(x)}{\max_{z \in \mathcal{P}} TT(z)}, \\ \overline{TP}(x) &= \frac{TP(x)}{\max_{z \in \mathcal{P}} TP(z)}, \\ \overline{SP}(x) &= \frac{SP(x)}{\max_{z \in \mathcal{P}} SP(z)}, \\ \forall x \in \mathcal{P}.\end{aligned}$$

The normalised Pareto front of solution approach h is defined as:

$$\overline{PF}_h = \{(\overline{TT}(x), \overline{TP}(x), \overline{SP}(x)) \mid x \in PF_h\}.$$

- (1) *Hypervolume Indicator (HV)*: The HV indicator quantifies the volume of the objective space dominated by a Pareto front relative to a reference point R (Zitzler and Thiele 1999). For our case, the reference point is $R = (1, 1, 1)$. For each solution approach $h \in H$, the HV is computed based on its normalised Pareto front $\overline{PF}_h$ as:

$$HV(\overline{PF}_h) = \text{vol} \left(\bigcup_{x \in \overline{PF}_h} [\overline{TT}(x), 1] \times [\overline{TP}(x), 1] \times [\overline{SP}(x), 1] \right).$$

Accordingly, $HV \in [0, 1]$ with a higher HV value indicating a better approximation of the true Pareto front, as a larger share of the objective space is dominated.

- (2) *Inverted Generational Distance (IGD)*: The IGD metric evaluates the average distance of reference Pareto-optimal solutions to an approximated Pareto front, thereby assessing convergence properties (Coello Coello and Reyes Sierra 2004; Zitzler et al. 2003). In our study, the reference front PF^* is defined as the set of all non-dominated solutions obtained across all approaches considered for evaluation. Let K denote the set of these reference solutions. The normalised reference Pareto front is defined as:

$$PF^* = \{(\overline{TT}(k), \overline{TP}(k), \overline{SP}(k)) \mid k \in K\}.$$

Then, the IGD for a given solution approach $h \in H$ is defined as:

$$IGD(\overline{PF}_h, PF^*) = \frac{1}{|PF^*|} \sum_{k \in K} \min_{x \in \overline{PF}_h} \|\bar{\mathbf{f}}(x) - \bar{\mathbf{f}}(k)\|_2,$$

where $\bar{\mathbf{f}}(\cdot) = (\overline{TT}(\cdot), \overline{TP}(\cdot), \overline{SP}(\cdot))$ and $\|\cdot\|_2$ denotes the Euclidean norm in the three-dimensional objective space. A smaller IGD value indicates a closer approximation of the reference front. As all objectives are normalised to $[0, 1]$, IGD values are bounded between 0 and $\sqrt{3}$.

- (3) *C Metric*: The C metric, also referred to as the coverage indicator, measures the fraction of solutions in one Pareto front that are weakly dominated by at least one solution in another Pareto front (Zitzler et al. 2003).

For two Pareto fronts $\overline{PF}_A$ and $\overline{PF}_B$, it is defined as

$$C(\overline{PF}_A, \overline{PF}_B) = \frac{|\{\bar{\mathbf{f}}_b \in \overline{PF}_B \mid \exists \bar{\mathbf{f}}_a \in \overline{PF}_A : \bar{\mathbf{f}}_a \preceq \bar{\mathbf{f}}_b\}|}{|\overline{PF}_B|},$$

where $\bar{\mathbf{f}}_a \preceq \bar{\mathbf{f}}_b$ indicates that $\bar{\mathbf{f}}_a$ weakly dominates $\bar{\mathbf{f}}_b$ in all objectives. The C metric takes values in $[0, 1]$, with higher values indicating that a larger fraction of solutions in $\overline{PF}_B$ are weakly dominated by solutions in $\overline{PF}_A$.

4.3. Hyperparameter tuning

To select the hyperparameters for our algorithm, we conducted experiments on subsets of small and large instances, as described in Section 4.1. The first hyperparameter is the selection method in Algorithm 1. We chose between random selection of a Pareto point (*SelectRnd*) or selection from a subset of the best solutions according to the sampled weight vector $\omega^{rnd,o}$ (*SelectBest*). The second hyperparameter is the destruction intensity (*NumDestr*) included in Algorithm 3. Given the variation in the number of operations, we differentiated the values between small and large instances, adopting value ranges for job destruction from extant literature and scaling them by the number of machines (Ruiz, Pan, and Naderi 2019). We tested the following hyperparameter values:

- Selection method: {*SelectRnd*; *SelectBest*}
- Destruction intensity (*NumDestr*):
 - Small instances: {5; 10; 15}
 - Large instances: {60; 80; 100}

We evaluated the performance of the hyperparameters based on HV and IGD, which provide stable, absolute quality indicators for single Pareto fronts. Since the C metric is inherently pairwise and depends on the chosen comparison front, it is not well suited for hyperparameter tuning. To determine the significance of hyperparameter effects, we conducted a two-way analysis of variance (ANOVA), followed by Tukey's Honestly Significant Difference (HSD) post-hoc test (Montgomery 2017). The ANOVA evaluates the main effect of each factor and their interaction effect on the mean performance. The ANOVA tables for both metrics are provided in the Appendix (Table A1). The analysis revealed significant main effects ($p \leq 0.05$) only for the IGD metric, specifically for the destruction intensity in small instances and the selection method in large instances.

Subsequently, we performed Tukey's HSD test to identify significant differences between specific hyperparameter levels while controlling for the family-wise error rate. Detailed results are reported in the Appendix (Table A2). For small instances, the test indicated that *NumDestr* = 15 yields significantly better performance than *NumDestr* = 5, with no significant difference for *NumDestr* = 10. Accordingly, we selected *NumDestr* = 15 as destruction intensity for small instances. For large instances, *SelectBest* significantly outperforms *SelectRnd*, which was therefore chosen as the selection method. For all non-significant factors (destruction intensity in large instances; selection method in small instances), we selected the hyperparameter level that yielded the best average performance metrics, which coincided for both HV and IGD in our experiments. Thus, we set *NumDestr* = 100 for large instances and *SelectRnd* for small instances.

4.4. Computational performance of the exact and heuristic methods

We analyze the solution status of the exact models in Section 4.4.1 and compare the performance of the MOIG, MP, and CP including the metrics (HV, IGD, C metric) in Section 4.4.2. Furthermore, we evaluate the inter-run stability and performance of the metaheuristic in Section 4.4.3.

4.4.1. Performance of the MP and CP formulations

We implemented the MP and CP formulations as described in Sections 2.2 and 2.3. Table 5 summarises the computational results of the exact formulations. The results are grouped by small and large instances, the number of jobs $n = \{5, 8, 10, 20, 50\}$, and the solution status. We differentiate three categories: (i) *Solved*, where all sampling points were solved optimally, (ii) *Solved w/ gap*, where one or more sampling points were either not started due to the global time limit or solved with an optimality gap due to the local time limit, and (iii) $|\text{ParetoSet}| = 0$, where no feasible solution was obtained for any sampling point within the global time limit. We report the number of instances (*Inst.*) and the average total runtime (*Time*) per category for both formulations. For the MP formulation, we further distinguish between the determination of objective ranges (including average runtime, percentage of optimally solved instances, and average gap) and the subsequent application of the AUGMECON method (including average percentage of sampling points that were solved optimally, solved with a gap, and not started due to the global time limit). For the CP formulation, we present the percentage of sampling points that were solved optimally, solved with a gap,

Table 5. Computational results of MP and CP grouped by number of jobs n and by the status, including instances (i) with all sampling points optimally solved (**Solved**), (ii) with (parts of) sampling points not started or solved with gap (**Solved w/ gap**), and (iii) no solution for any sampling points (**ParetoSet = 0**).

		MP										CP									
		Overall					Obj. Range Det.					AUGMECON					Overall (weighted sum)				
n	Status	Inst.	Time	Time	Solved	Solved	Gap	Solved*	W/ gap*	Not started*	Gap	Inst.	Time	Gap	Solved*	W/ gap*	Not started*				
Small ($m = \{3; 5\}$)	5	Solved	50	19s	4s	100%	0.00%	0.00%	100%	0%	0%	0.00%	50	98s	0.00%	100%	0%	0%			
		Solved w/ gap	4	1800s	295s	100%	0.00%	0.00%	8%	10%	82%	51.69%	14	1643s	19.40%	29%	5%	66%			
		ParetoSet = 0	10	1800s	1800s	84%	8.21%						0								
	8	Solved	13	236s	136s	100%	0.00%	0.00%	100%	0%	0%	0.00%	8	226s	0.01%	100%	0%	0%			
		Solved w/ gap	3	1800s	872s	100%	0.00%	0.00%	12%	30%	58%	75.00%	56	1584s	32.13%	18%	6%	76%			
		ParetoSet = 0	48	1800s	1800s	64%	34.78%						0								
	10	Solved	0	-	-	-	-	-	-	-	-	-	0	-	-	-	-	-			
		Solved w/ gap	1	1800s	334s	100%	0.00%	0.00%	9%	45%	46%	73.33%	64	1541s	48.28%	12%	7%	81%			
		ParetoSet = 0	63	1800s	1800s	63%	36.72%						0								
Large ($m = \{10; 20\}$)	10	Solved	5	1099s	1081s	100%	0%	0%	100%	0%	0%	0.00%	0	-	-	-	-	-			
		Solved w/ gap	0	-	-	-	-	-	-	-	-	-	96	1607s	31.08%	17%	8%	75%			
		ParetoSet = 0	91	1800s	1800s	59%	39.60%						0								
	20	Solved	0	-	-	-	-	-	-	-	-	-	0	-	-	-	-	-			
		Solved w/ gap	0	-	-	-	-	-	-	-	-	-	0								
		ParetoSet = 0	96	1800s	1800s	25%	81.87%						96	1613s	56.42%	6%	6%	88%			
	50	Solved	0	-	-	-	-	-	-	-	-	-	0	-	-	-	-	-			
		Solved w/ gap	0	-	-	-	-	-	-	-	-	-	0								
		ParetoSet = 0	96	1800s	1800s	0%	0.00%						96	1504s	82.47%	2%	7%	91%			
Total/ Avg.		Solved w/ gap	0	-	-	-	-	-	-	-	-	-	0								
		ParetoSet = 0	96	1800s	1800s	0%	0.00%						58	115s	0.00%	100%	0%	0%			
		Solved w/ gap	8	1800s	516s	100%	0.00%	0.00%	10%	22%	68%	63.14%	422	1573s	50.89%	82%	7%	11%			
	ParetoSet = 0	404	1800s	1800s	39%	43.21%						0									

* Average percentage of sampling points that are solved optimally (**solved**), solved with gap (**w/ gap**), or not started due to timeout (**not started**).

and not started within the global time limit, as well as the average optimality gap across all sampling points.

For the smallest class ($n = 5$), both MP and CP exhibit strong performance, with 50 out of 64 instances solved optimally and low runtime. However, as the instance size increases, the performance of both approaches declines noticeably. Of the 64 instances at $n = 8$ the MP formulation solves 13 instances to optimality and three instances with gap, while 48 instances stop during objective range determination, precluding exploration of the Pareto set. For $n = 10$, the MP formulation solved one of the small instances with gap and five of the large instances to optimality, due to a favourable solution structure. However, the objective range determination timed out for the remaining 63 small and 91 large instances. No solutions were found for the next-largest instances with $n = \{20; 50\}$, suggesting that the MP formulation is not well suitable for solving large problem instances.

In contrast, the CP formulation maintains a higher degree of progress for larger instances, as it successfully evaluates parts of the sampling points for all small and large instances. For $n = 8$, CP is capable of solving six instances to optimality. Of the remaining 56 instances, it solved 18% of the sampling points and analysed an additional 6% with an optimality gap. As instance size increases, the percentage of optimally solved sampling points decreases to 12% for $n = 10$ (small instances), 17% for $n = 10$ (large instances), 6% for $n = 20$, and 2% for $n = 50$. Concurrently, the percentage of sampling points that could not be analysed due to a global time-out increases to over 90% at $n = 50$, indicating a clear performance limit of CP for large instances.

Comparing the two methods across all sizes, CP demonstrates greater progress in solving for more complex instances ($n \geq 8$), while MP is primarily competitive for the smallest instance sizes. Applying the weighted sum method, CP processes more sampling points and explores a wider range of trade-offs. Concurrently, the performance of MP with the AUGMECON method deteriorates more rapidly as the instance size increases, primarily due to the computational effort required to determine the objective ranges.

4.4.2. Comparison of MO-IG, MP, and CP

We implemented the MO-IG as described in Sections 3 and 4.3. To account for stochastic fluctuations, we executed the MO-IG three times. Based on the performance metrics described in Section 4.2 we compared the computational results of MO-IG, MP, and CP, as shown in Table 6. The Pareto point results of all solution approaches for all small and large instances are published in the Online Appendix. We normalised TT , TP , and SP based on the maximum values of each run and

Table 6. Comparison of performance metrics (HV, IGD, C Metric) for the exact formulations (MP, CP) and the MO-IG, grouped by the number of jobs (n) and machines (m), including additional metrics on the MO-IG performance: (i) $HV \prec MP \& CP / IGD \succ MP \& CP$: Number of instances in which HV/IGD of MO-IG is worse than that of MP or CP; (ii) $IGD > 0.001\%$: Number of instances with $IGD(MO - IG) > 0.001\%$.

n	m	Inst.	MP (all %)				CP (all %)				MO-IG			
			HV	IGD	C(MP,IG)	C(IG,MP)	HV	IGD	C(CP,IG)	C(IG,CP)	HV (%)	IGD (%)	HV $\prec$ MP&CP	IGD $\succ$ MP&CP
5	3	54	68.10	12.82	100.00	68.96	67.60	7.09	100.00	65.99	74.36	0.00	0	0
		29	41.85	23.73	100.00	45.43	41.00	12.95	100.00	39.63	53.33	0.00	0	0
8	5	25	98.56	0.17	100.00	96.25	98.46	0.29	100.00	96.56	98.75	0.00	0	0
		16	43.04	30.04	95.83	41.56	36.48	22.68	100.00	31.45	59.08	0.01	0	3
10	3	9	39.60	21.06	100.00	60.01	35.75	11.96	100.00	48.43	46.83	0.00	0	0
		7	47.46	41.59	90.48	17.84	37.43	36.46	100.00	9.61	74.82	0.03	0	3
3	1	1	57.15	17.97	100.00	0.55	95.42	8.44	100.00	1.66	98.25	0.00	0	0
		1	57.15	17.97	100.00	0.55	95.42	8.44	100.00	1.66	98.25	0.00	0	0
Total/Avg.		71	62.30	16.77	99.06	61.82	60.98	10.62	100.00	57.30	71.25	0.00	0	3

Table 7. Comparison of performance metrics (HV, IGD, C Metric) between the CP formulation and the MO-IG, grouped by the number of jobs (n) and machines (m), including additional metrics on the MO-IG performance: (i) $HV \prec CP / IGD \succ CP$: Number of instances in which HV/IGD of MO-IG is worse than that of CP. (ii) $IGD > 0.001\%$: Number of instances with $IGD(MO - IG) > 0.001\%$.

		CP (all in %)					MO-IG				
n	m	Inst.	HV	IGD	C(CP,IG)	C(IG,CP)	HV (%)	IGD (%)	HV $\prec$ CP	IGD $\succ$ CP	IGD > 0.001%
5		64	70.04	8.79	99.79	56.28	77.79	0.00	0	0	0
	3	32	45.37	13.40	100.00	36.27	57.22	0.00	0	0	0
	5	32	94.70	4.19	99.58	76.30	98.36	0.00	0	0	0
8		64	59.69	35.99	95.24	8.64	81.04	0.00	0	0	6
	3	32	50.52	36.16	96.61	14.60	71.39	0.01	0	0	3
	5	32	68.87	35.83	93.86	2.69	90.69	0.00	0	0	4
10		64	55.41	48.59	96.63	0.21	83.58	0.00	0	0	3
	3	32	44.68	55.20	99.55	0.19	77.68	0.00	0	0	0
	5	32	66.15	41.98	93.71	0.23	89.49	0.00	0	0	2
10		96	60.00	36.11	91.91	3.78	95.74	0.01	0	0	18
	10	48	71.85	34.39	92.66	0.17	96.45	0.00	0	0	7
	20	48	48.16	37.82	91.16	7.40	95.02	0.01	0	0	11
20		96	12.11	81.00	89.50	0.00	91.42	0.01	0	0	20
	10	48	16.24	76.32	88.41	0.00	89.12	0.00	0	0	10
	20	48	7.98	85.68	90.58	0.00	93.72	0.01	0	0	10
50		96	2.89	74.29	94.88	0.00	87.91	0.01	0	0	11
	10	48	3.52	72.48	97.80	0.00	87.22	0.00	0	0	3
	20	48	2.25	76.10	91.96	0.00	88.59	0.03	0	0	8
Total/Avg.		480	39.69	50.73	94.14	9.44	87.33	0.01	0	0	58

computed the HV of each normalised Pareto front relative to the reference point $R = (1, 1, 1)$. Furthermore, we calculated the IGD for each MO-IG run relative to a reference Pareto front PF^* from the non-dominated values of all solution approaches. Finally, for MP and CP we determined the fraction of Pareto points that are weakly dominated by ($C(MP, IG)$; $C(CP, IG)$) and that weakly dominate ($C(IG, MP)$; $C(IG, CP)$) the Pareto points of each MO-IG run.

To ensure comparability, we only considered instances for which all solution approaches yielded at least one sampling point. First, Table 6 compares all solution approaches (MP, CP, MO-IG). Since MP only found sampling points for five of the 288 large instances, this comparison focuses on small instances, including 71 instances with assessable results across all approaches. Second, Table 7 compares only CP and MO-IG, thereby allowing for the inclusion of all 480 small and large instances. Because deviations between the MO-IG runs were negligible, the tables report mean values over all three runs. The results are grouped by the number of jobs n and number of machines m .

Our results demonstrate that our MO-IG algorithm consistently outperforms both exact methods across all small instances with respect to all performance metrics, HV, IGD, and C metric. As shown in Table 6, MO-IG achieves the highest average HV (71%), surpassing MP (62%) and CP (61%). The HV values vary considerably across different combinations of n and m , reflecting the strong dependency of the HV on the geometric structure of the respective Pareto front. The superiority of MO-IG is confirmed by the IGD values: 0% for MO-IG, 17% for the MP formulation, and 11% for the CP

formulation. The C metrics $C(MP, IG)$ and $C(CP, IG)$ are close to 100% in most cases in Table 6, indicating that the Pareto fronts produced by MO-IG weakly dominate those of the exact formulations. Conversely, the metrics $C(IG, MP)$ and $C(IG, CP)$ remain substantially lower, showing that MP and CP only cover subsets of the MO-IG front. To further quantify comparative performance, we count the number of instances where MO-IG outperforms both exact methods according to HV and IGD (i.e. $HV \prec MP \& CP$; $IGD \succ MP \& CP$). Across the instances in Table 6, this holds for all cases, highlighting the consistently superior performance of MO-IG. Additionally, we report the number of instances where the IGD of MO-IG exceeds a threshold of 0.001%. This occurs in only three out of 71 instances, indicating consistent convergence to the reference front. For instances with $n = 5$, both the MP and CP proved optimality for most instances, while none of the MO-IG instances exceeded an IGD of 0.001%. This confirms that the MO-IG successfully recovered the optimal objective values.

When comparing the metrics of MP and CP in Table 6 directly, MP generally yields slightly higher values of HV and C metric, suggesting a broader exploration of the objective space, whereas the slightly lower IGD values of CP indicate a closer approximation to the reference front. However, the differences are minimal, indicating that the weighted sum approach in CP is empirically as effective as the ε -constraint approach in MP for recovering Pareto fronts of the investigated problem. Nevertheless, neither exact method approaches the performance of MO-IG.

The comparison between the results of CP and MO-IG in Table 7 confirms the superiority of MO-IG for

both small and large instances. Across all instances, MO-IG consistently delivers higher HV (87% vs. 40%) and lower IGD values (0% vs. 51%). Only 58 of 480 instances exhibit an IGD above 0.001%. Furthermore, $C(\text{CP}, \text{IG})$ consistently remains above 85% across all categories, indicating high coverage of the CP results through the metaheuristic.

Our computational study shows that the MO-IG heuristic clearly outperforms the exact MP and CP formulations. The specialised search logic of the MO-IG allows for a more efficient exploration of the search space than the exact models, yielding high-quality solutions. Across all instances, MO-IG consistently achieves higher HV and smaller IGD values, indicating both broader coverage and closer proximity to the reference Pareto front. On average, the performance gap between the metaheuristic algorithm and the exact formulations increases with larger instance sizes for all three metrics, emphasising that the relative advantage of MO-IG becomes more pronounced for larger problem sizes. These results underline the efficiency and robustness of the MO-IG for MO-DFSP-TS.

4.4.3. Performance of the MO-IG

To evaluate the inter-run stability and to determine the limitations of the MO-IG, we calculated the performance metrics HV and IGD for the all instances across three independent runs. Additionally, we report the average deviation of HV and IGD between runs (ΔHV and ΔIGD), the number of Pareto-efficient solutions found, and the number of iterations executed within the time limit. A deterioration in these metrics indicates greater deviation between runs and a decline in solution quality. The results are summarised in Table 8. Overall, the deviations between runs are small for both HV (below 2.6%)

and IGD (below 3.9%), indicating that the algorithm produces stable results across independent executions.

Table 8 further reveals that instance size strongly affects performance and inter-run stability. The number of Pareto-efficient solutions initially increases up to the instance size of $n = 20$ and $m = 10$, likely due to a higher number of feasible solutions. However, for larger instances, the number of Pareto solutions decreases considerably. A similar trend is reflected in the deterioration of HV, ΔHV , IGD, and ΔIGD at an increasing instance size.

A key observation is the exponential reduction in iterations as instance size grows. While instances with $m = 3$ and $n = 5$ allow more than 35 million iterations and instances with $m = 10$ and $n = 10$ still more than 1.6 million iterations, the largest instances with $m = 20$ and $n = 50$ permit fewer than 5000 iterations. This drastic reduction in search budget limits the algorithm's ability to effectively apply its destruction–repair and local search operators. Although the correlation between performance metrics and number of iterations is only moderate, the extreme decline in iteration count suggests a severe time-budget bottleneck: for large instances, the algorithm cannot explore the search space sufficiently, which likely contributes to the reduced number of Pareto solutions and the deterioration of HV and IGD.

In summary, MO-IG is able to solve substantially larger instances than the exact methods and demonstrates stable behaviour across independent runs. As inter-factory transportation and production speed variation greatly increase the complexity, an instance size of $n = 50; m = 20$ effectively represents a computational limit for large instances.

Table 8. Performance metrics (HV, IGD, number of iterations and number of Pareto points ($|ParetoSet|$)) across three runs of the MO-IG for all instances, grouped by number of machines (m) and jobs (n).

m	n	HV (%)	ΔHV (%)	IGD (%)	ΔIGD (%)	$ ParetoSet $	Iterations
3		56.52	0.00	0.09	0.07	360	35,418,955
	5	52.82	0.00	0.06	0.04	43	71,725,906
	8	53.78	0.00	0.06	0.02	332	22,655,344
5	10	62.97	0.00	0.15	0.13	704	11,875,615
		73.65	0.08	0.51	0.37	471	22,673,649
	5	92.37	0.00	0.04	0.02	31	48,436,146
	8	62.22	0.04	0.30	0.24	484	12,275,531
	10	66.35	0.19	1.21	0.85	897	7,309,271
10		61.72	0.76	2.89	1.21	1,467	730,829
	10	70.66	0.45	2.37	1.51	795	1,822,799
	20	67.02	1.16	3.23	1.36	2,209	354,743
	50	47.49	0.68	3.07	0.77	1,396	14,944
		68.01	1.83	4.83	2.41	1,153	306,963
20	10	70.24	2.54	5.47	3.85	596	769,847
	20	74.70	1.65	4.38	2.21	2,009	146,049
	50	59.10	1.31	4.63	1.17	853	4,993
Avg.		64.95	0.79	2.44	1.17	952	11,929,858

ΔHV : Average deviation of the HV per instance between runs.

ΔIGD : Average deviation of the IGD per instance between runs.

4.5. Impact of flexibility mechanisms

In this section, we analyze how inter-factory transportation and production speed variation influence both cost and total tardiness. Section 4.5.1 assesses the impact of the distance on inter-factory transportation and Section 4.5.2 evaluates the combined value of inter-factory transportation and production speed variation.

4.5.1. Impact of distance on inter-factory transportation

To gain insight into the sensitivity and limits of inter-factory transportation, we analysed the impact of transportation distances on the resulting schedules for all large instances. In each instance, we use homogeneous distances $t_{kk'} \in \{5, 10, 20, 50\} \forall k \in K, \forall k' \in K$ for $k \neq k'$ and $t_{kk} = 0$. We evaluate the relevance of inter-factory transportation using three key metrics: the maximum number of transports, the maximum number of transports required to reduce total tardiness to 75%, and the median number of transports across all Pareto points. An increase in distance has two effects. First, a direct effect on the schedule, as higher distance values lead to longer job unavailability during transport. Second, an economic effect, as transport costs directly increase with higher distances.

As shown in Figure 3, the distance is a decisive factor for the utilisation of inter-factory transportation. In addition, the number of jobs directly impacts the metrics, particularly the maximum overall number of transports.

A large number of transports is scheduled at a distance of five, ranging from a maximum of 11 for $n = 10$ to 186 for $n = 50$. However, the number of transports decreases significantly at distances of 10 and 20. At a distance of 50, the number of transports is negligible, with a maximum of 11 transports for $n = 50$ and of one transport for $n = 10$.

In summary, these findings demonstrate that inter-factory transportation is a powerful lever for reducing tardiness at lower distances of 5, 10, and 20. However, it becomes inefficient for the majority of jobs at a distance of 50, limiting its utilisation to critical or edge cases.

4.5.2. Combined value of inter-factory transportation and production speed variation

Our objective in this subsection is twofold: (i) to evaluate the trade-offs between transportation and speed-ups in reducing total tardiness, and (ii) to derive managerial insights. Since the problem is highly complex and each instance exhibits a unique structure, we perform a detailed analysis of exemplary Pareto fronts for a curated subset of small and large instances, as introduced in Section 4.1. We selected a transportation distance of $t_{kk'} = 10 \forall k \in K, \forall k' \in K$, which still allows for the effective application of transports as demonstrated in Section 4.5.1. To simplify interpretation, we considered only two production speed levels: (1) base speed ($v = 1.0$) and (2) 10% speed-up ($v = 1.1$).

Figures 4 and 5 show the resulting Pareto fronts, with normalised total tardiness on the y -axis and the number

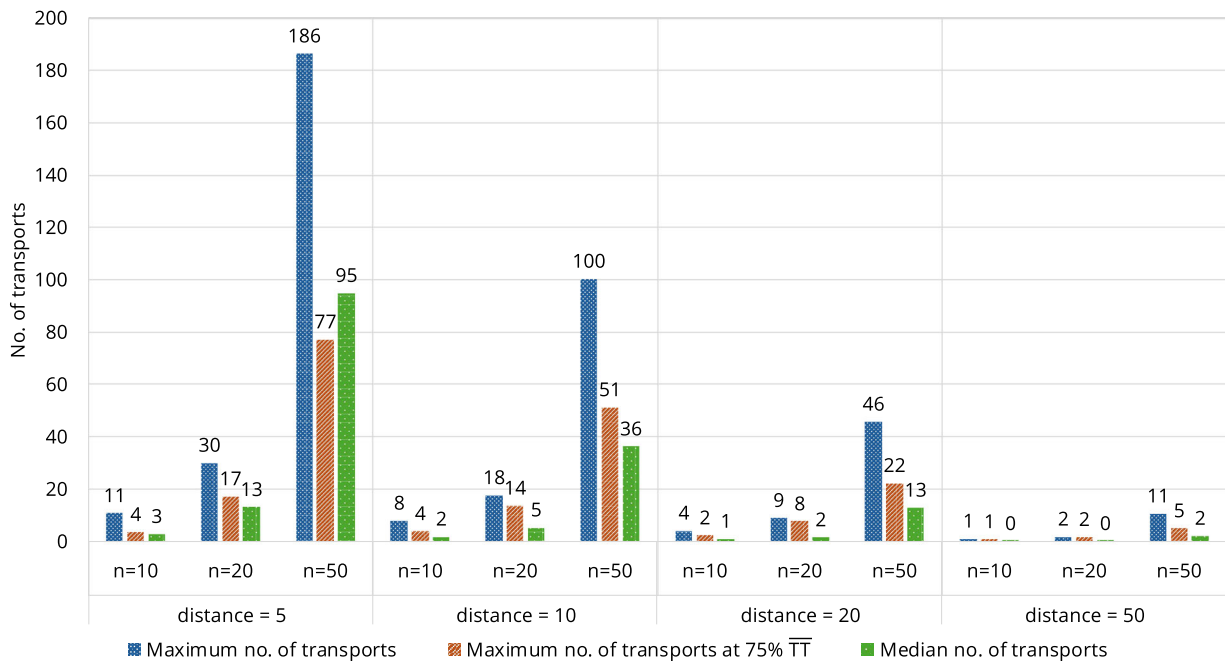


Figure 3. Impact of distance on maximum number of transports, maximum number of transports at $\overline{TT} = 75\%$, and median number of transports across all large instances, grouped by number of jobs n .

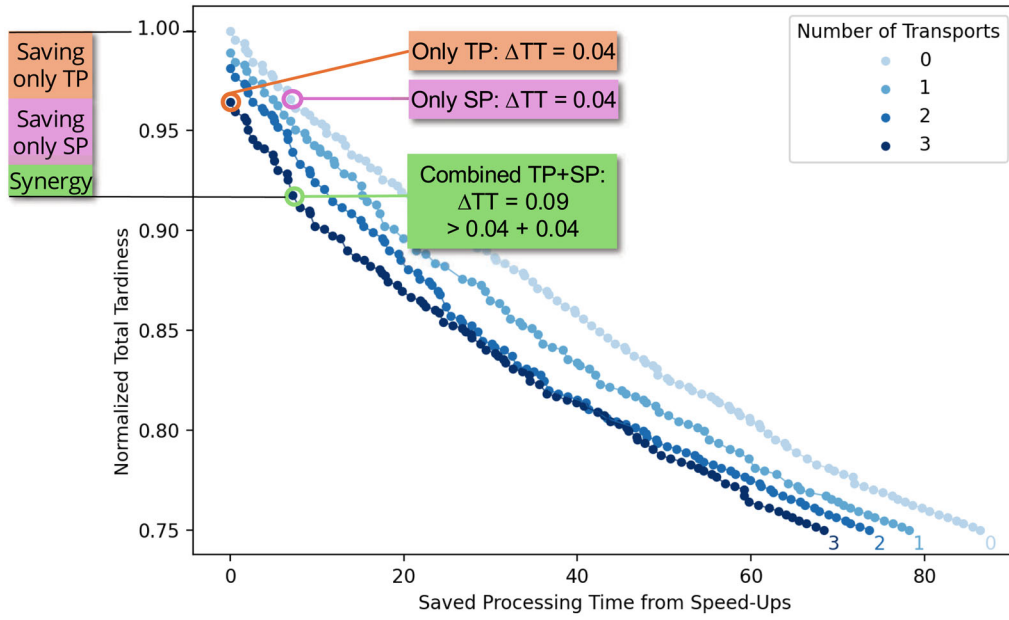


Figure 4. Pareto front for an instance with $f = 2$; $n = 10$; $m = 5$ showing the normalised total tardiness $\overline{TT}$ (y-axis), the saved processing time from speed-ups derived from $\mathcal{SP}$ (x-axis) and the number of transports derived from $\mathcal{TP}$ (labels).

of processing time units saved by speed-ups on the x -axis. We report processing time savings rather than the speed-up cost $\mathcal{SP}$ to enable a more direct interpretation of the x -axis. For a single speed level, there is a linear relationship between saved processing time and the value of $\mathcal{SP}$. This representation preserves the dominance relations between $\mathcal{TT}$, $\mathcal{TP}$, and $\mathcal{SP}$. The Pareto points are differentiated by the number of inter-factory transports, which are grouped into a limited number of color-coded classes. Within each class, we only include non-dominated solutions, and lines connect solutions with the highest transport counts, highlighting the marginal effect of additional speed-ups. To facilitate managerial insight, we focus on the region in which total tardiness is reduced from 100% to 75%. Further visualisations of these Pareto fronts can be found in the Online Appendix.

To investigate the value of inter-factory transportation, production speed variation, and their interaction, we recognise three possible effects of combined use:

- Exclusive use: No Pareto-efficient solution employs both inter-factory transportation and production speed variation, indicating that joint use does not provide an efficiency advantage.
- Additive joint use: Pareto-efficient solutions exist that combine inter-factory transportation and production speed variation, and the combined reduction in total tardiness exceeds that achieved by either mechanism alone. However, the total savings from combining both flexibility mechanisms do not exceed the sum of the individual savings.

- Synergistic joint use: Pareto-efficient solutions combining transportation and speed variation achieve a greater reduction in total tardiness than the sum of the individual effects, demonstrating a synergistic interaction between the two mechanisms.

Figure 4 highlights a region of the Pareto front in which there is synergy between inter-factory transportation and production speed variation. In this instance, introducing three transports reduces $\overline{TT}$ by approximately 4% (orange sampling point), while achieving a comparable reduction solely through speed-ups requires increasing the speed by 17 units (pink sampling point). Combining the effects of introducing three transports and accelerating the process times by 17 units reduces $\overline{TT}$ by 9% (green sampling point), which exceeds the sum of their isolated improvements. This indicates a synergy between inter-factory transportation, which reorganises the schedule to enhance machine utilisation, and speed variation, which directly compensates for bottlenecks.

The exemplary instances in Figures 4 and 5 demonstrate that both flexibility options effectively reduce total tardiness. Exclusive use of only one flexibility option is observed in relatively few cases across the analysed instances, suggesting that joint use is generally advantageous. This is particularly evident in Figures 4 and 5(a,c–f), in which, on average, 86% of Pareto-efficient solutions combine transportation and speed-ups. Moreover, all of these instances exhibit regions of the Pareto front with synergy, indicating that the combined use of

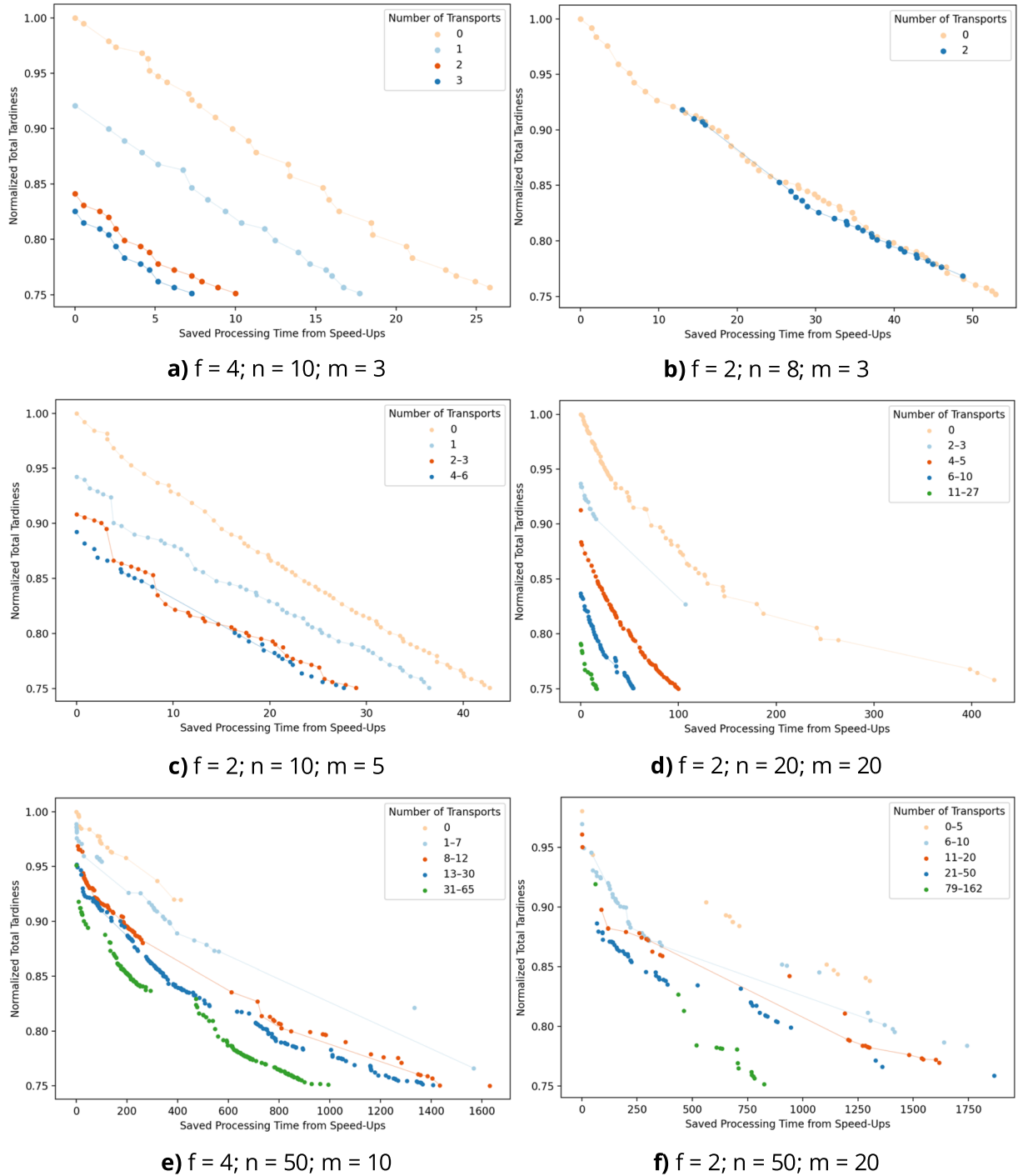


Figure 5. Pareto fronts of small exemplary instances in (a–c) and large exemplary instances in (d–f) showing the normalised total tardiness $\overline{TT}$ (y-axis), the saved processing time from speed-ups derived from $\mathcal{SP}$ (x-axis) and the number of transports derived from $\mathcal{TP}$ (labels).

both flexibility options yields additional benefits beyond their isolated application. Figure 5(b) presents an exception where only few combined solutions exist and no synergy is observed. In this instance, inter-factory transportation provides limited additional value, as it affects the schedule only at specific positions while introducing additional transportation delays.

In addition to analyzing their interaction, we examine the effects of each flexibility option individually. The benefit of additional transportation is reflected by the vertical distance between consecutive transportation lines. The value of speed-ups is encoded by the slope of each line, with steeper slopes reflecting greater reduction in tardiness per speed-up. General statements cannot be made

about the effective range of transports, speed-ups, or their combination across Figures 4 and 5 because the effect depends heavily on the instance-specific characteristics of the slacks and bottlenecks. Inter-factory transportation plays a particularly important role in reducing total tardiness in Figure 5(a,c,d), with 10% to 20% of the total tardiness reduction achievable solely through transportation. Across all instances, the marginal reduction in total tardiness is highest for the first few transports and decreases sharply as additional transports are introduced. Overall, transporting up to 11% of the operations is effective across instances within the considered section of the Pareto front. Speed-ups are beneficial across all instances, as they can be incorporated flexibly without disrupting the schedule. In contrast to transports, speed-ups exhibit more gradual diminishing returns, providing smaller but more consistent improvements. Speed variation plays a particularly significant role in Figure 5(d,f), with more than 70% of operations receiving a speed-up at the maximum. On average, up to 31% of operations in small instances and up to 57% of operations in large instances are subject to a speed-up, across the curated subset.

Overall, our findings indicate that the most substantial performance improvements arise from a coordinated application of inter-factory transportation and production speed variation. The results do not include any strictly exclusive pattern. Instead, the combined use of both flexibility options consistently yields the largest reduction in total tardiness across all instances. While production speed variation is typically more flexible than inter-factory transportation, transportation also generates value at specific structural bottlenecks within the schedule. Although both flexibility options are effective when used in isolation, their joint use frequently unlocks additional benefits in the analysed instances. From a managerial perspective, this suggests that production speed variation and inter-factory transportation should not be treated as substitutes but as complementary decision levers. Speed-ups provide continuous, fine-grained control, whereas transportation enables discrete structural adjustments. Leveraging both mechanisms in a coordinated manner allows decision makers to better align the production structure and mitigate remaining bottlenecks, thereby achieving more effective customer-oriented schedules.

5. Conclusion and outlook

In this paper, we introduced the MO-DFSP-TS. By jointly considering total tardiness and the costs of transportation and speed variation, we incorporated a customer-oriented objective and two relevant flexibility mechanisms into a novel multi-objective scheduling

framework. To solve the proposed problem, we developed mathematical programming and constraint programming formulations as exact methods and introduced a tailored multi-objective iterated greedy metaheuristic. The computational results demonstrate that the metaheuristic is able to efficiently generate high-quality Pareto fronts and clearly outperforms the exact approaches. Beyond algorithmic performance, our analysis provides insight into the value of inter-factory transportation, production speed variation, and its combined use in distributed flow shops. The results show that while both flexibility options are suitable for reducing total tardiness, their combined use is particularly effective and can yield additional benefits, highlighting their complementary nature.

Future research could explore alternative heuristic, metaheuristic, or hybrid solution approaches to further address the computational challenges of distributed flow shops with transportation and speed variation. While our model leverages flexibility mechanisms specifically to meet individual due dates, future studies could explore these trade-offs in a makespan-oriented context. Furthermore, additional objectives such as workload balance between factories or explicit environmental costs could be integrated into the multi-objective framework. Moreover, incorporating uncertainties related to transportation times, machine availability, or demand represents a promising direction to further enhance the practical relevance of distributed scheduling models.

Author contributions

CRedit: **Lydia Hilarius:** Conceptualization, Data curation, Formal analysis, Investigation, Methodology, Project administration, Software, Validation, Visualization, Writing – original draft, Writing – review & editing; **Udo Buscher:** Conceptualization, Investigation, Methodology, Supervision, Validation, Writing – review & editing; **Tristan Becker:** Conceptualization, Formal analysis, Investigation, Methodology, Software, Supervision, Validation, Writing – review & editing

Disclosure statement

No potential conflict of interest was reported by the author(s).

Notes on contributors



Lydia Hilarius is a research assistant at the Junior Professorship in Management Science at Technische Universität Dresden, Germany, where she also received her Diploma in 2017. Her research focuses on the development and application of exact and heuristic optimisation approaches for scheduling and network planning problems. In her Ph.D. research, she investigates distributed flow shop scheduling and the design of cloud infrastructure and heat supply networks.



Udo Buscher is a professor of Industrial Management at Technische Universität Dresden, Germany, where he teaches courses on operations management and operations research. He studied at the University of Göttingen, Germany, received a Ph.D. from Technische Universität Dresden, and finished his habilitation at the University of Würzburg, Germany. His research interests include production planning and control, supply chain management, and scheduling with a strong focus on operations research, decision science, applied algorithms, and game theory. He carried out numerous projects with companies in which OR methods play a central role. His research results are published in leading international academic journals.



Jun.-Prof. Dr. rer. oec. Tristan Becker holds the Junior Professorship of Management Science at Technische Universität Dresden, Germany. He studied at Ruhr University Bochum, where he also received his Ph.D. in 2019, and subsequently held a postdoctoral position at RWTH Aachen University. His research interests include scheduling, location analysis, and heuristic optimisation methods, with a strong focus on operations research and decision support for complex planning problems. His research has been published in leading international academic journals. He has received several awards for his research, including the Young Researchers Award of the German Operations Research Society (GOR).

Data availability statement

The data that is used in the computational study, along with supplementary information, has been published in the online appendix: DOI: 10.17632/9tv5svjbg3.1

References

- Becker, T., J. Neufeld, and U. Buscher. 2025. "The Distributed Flow Shop Scheduling Problem with Inter-factory Transportation." *European Journal of Operational Research* 322 (1): 39–55. <https://doi.org/10.1016/j.ejor.2024.10.026>.
- Cai, S., K. Yang, and K. Liu. 2018. "Multi-objective Optimization of the Distributed Permutation Flow Shop Scheduling Problem with Transportation and Eligibility Constraints." *Journal of the Operations Research Society of China* 6 (3): 391–416. <https://doi.org/10.1007/s40305-017-0165-3>.
- Chang, X., X. Jia, and J. Ren. 2025. "A Reinforcement Learning Enhanced Memetic Algorithm for Multi-objective Flexible Job Shop Scheduling toward Industry 5.0." *International Journal of Production Research* 63 (1): 119–147. <https://doi.org/10.1080/00207543.2024.2357740>.
- Chen, T.-L., C.-Y. Cheng, and Y.-H. Chou. 2020. "Multi-objective Genetic Algorithm for Energy-Efficient Hybrid Flow Shop Scheduling with Lot Streaming." *Annals of Operations Research* 290 (1): 813–836. <https://doi.org/10.1007/s10479-018-2969-x>.
- Cheng, L., L. Wang, J. Cai, K. Hu, Y. Xiong, and Q. Xia. 2025. "An Effective Biogeography-Based Optimization Algorithm for Multi-objective Green Scheduling of Distributed Assembly Permutation Flowshop Scheduling Problem." *Computers & Operations Research* 183:107158. <https://doi.org/10.1016/j.cor.2025.107158>.
- Ciavotta, M., G. Minella, and R. Ruiz. 2013. "Multi-objective Sequence Dependent Setup times Permutation Flowshop: A New Algorithm and a Comprehensive Study." *European Journal of Operational Research* 227 (2): 301–313. <https://doi.org/10.1016/j.ejor.2012.12.031>.
- Coello Coello, C. A., and M. Reyes Sierra. 2004. "A Study of the Parallelization of a Coevolutionary Multi-objective Evolutionary Algorithm. In *MICAI 2004: Advances in Artificial Intelligence*, edited by R. Monroy, G. Arroyo-Figueroa, L. E. Sucar, and H. Sossa, 688–697, Berlin, Heidelberg: Springer Berlin Heidelberg.
- Datta, D., and J. R. Figueira. 2012. "Some Convergence-Based M-Ary Cardinal Metrics for Comparing Performances of Multi-objective Optimizers." *Computers & Operations Research* 39 (7): 1754–1762. <https://doi.org/10.1016/j.cor.2011.10.013>.
- Ding, J., S. Schulz, L. Shen, U. Buscher, and Z. Lü. 2021. "Energy Aware Scheduling in Flexible Flow Shops with Hybrid Particle Swarm Optimization." *Computers & Operations Research* 125:105088. <https://doi.org/10.1016/j.cor.2020.105088>.
- Fathollahi-Fard, A. M., L. Woodward, and O. Akhrif. 2024. "A Scenario-Based Robust Optimization Model for the Sustainable Distributed Permutation Flow-Shop Scheduling Problem." *Annals of Operations Research*. <https://link.springer.com/article/10.1007/s10479-024-05940-7#citeas>.
- Feng, X., F. Zhao, G. Jiang, T. Tao, and X. Mei. 2024. "A Tabu Memory Based Iterated Greedy Algorithm for the Distributed Heterogeneous Permutation Flowshop Scheduling Problem with the Total Tardiness Criterion." *Expert Systems with Applications* 238:121790. <https://doi.org/10.1016/j.eswa.2023.121790>.
- Fernandez-Viagas, V., and J. M. Framinan. 2015. "NEH-based Heuristics for the Permutation Flowshop Scheduling Problem to Minimise Total Tardiness." *Computers & Operations Research* 60:27–36. <https://doi.org/10.1016/j.cor.2015.02.002>.
- Fu, Y., Y. Hou, Z. Chen, X. Pu, K. Gao, and A. Sadollah. 2022. "Modelling and Scheduling Integration of Distributed Production and Distribution Problems via Black Widow Optimization." *Swarm and Evolutionary Computation* 68:101015. <https://doi.org/10.1016/j.swevo.2021.101015>.
- Gao, Q., H. Hu, J. Liu, C. Zhuang, H. Ding, and H. Li. 2024. "Distributed Assembly Shop Scheduling Problem for Complex Products considering Multiskilled Worker Assignment and Transportation Time." *International Journal of Production Research* 64 (13): 5486–5508.
- Ghorbani Saber, R., and M. Ranjbar. 2022. "Minimizing the Total Tardiness and the Total Carbon Emissions in the Permutation Flow Shop Scheduling Problem." *Computers & Operations Research* 138:105604. <https://doi.org/10.1016/j.cor.2021.105604>.
- Han, Z., B. Zhang, H. Sang, C. Lu, L. Meng, and W. Zou. 2025. "Optimising Distributed Heterogeneous Flowshop Group Scheduling Arising from Pcb Mounting: Integrating Construction and Improvement Heuristics." *International Journal of Production Research* 63 (5): 1753–1778. <https://doi.org/10.1080/00207543.2024.2390979>.

- He, X., Q.-K. Pan, L. Gao, J. S. Neufeld, and J. N. Gupta. 2024. "Historical Information Based Iterated Greedy Algorithm for Distributed Flowshop Group Scheduling Problem with Sequence-Dependent Setup times." *Omega* 123:102997. <https://doi.org/10.1016/j.omega.2023.102997>.
- He, X., Q.-K. Pan, L. Gao, J. S. Neufeld, and J. N. Gupta. 2025. "Minimising Makespan and Total Tardiness for the Flowshop Group Scheduling Problem with Sequence Dependent Setup times." *European Journal of Operational Research* 324 (2): 436–453. <https://doi.org/10.1016/j.ejor.2025.02.009>.
- Hou, Y., H. Wang, Y. Fu, K. Gao, and H. Zhang. 2023. "Multi-objective Brain Storm Optimization for Integrated Scheduling of Distributed Flow Shop and Distribution with Maximal Processing Quality and Minimal Total Weighted Earliness and Tardiness." *Computers & Industrial Engineering* 179:109217. <https://doi.org/10.1016/j.cie.2023.109217>.
- Khare, A., and S. Agrawal. 2021. "Effective Heuristics and Meta-heuristics to Minimise Total Tardiness for the Distributed Permutation Flowshop Scheduling Problem." *International Journal of Production Research* 59 (23): 7266–7282. <https://doi.org/10.1080/00207543.2020.1837982>.
- Li, H., L. Gao, Q. Fan, X. Li, and B. Han. 2025. "An End-to-end Decentralised Scheduling Framework Based on Deep Reinforcement Learning for Dynamic Distributed Heterogeneous Flowshop Scheduling." *International Journal of Production Research* 63 (12): 4368–4388. <https://doi.org/10.1080/00207543.2024.2449240>.
- Li, H., X. Li, and L. Gao. 2021. "A Discrete Artificial Bee Colony Algorithm for the Distributed Heterogeneous No-wait Flowshop Scheduling Problem." *Applied Soft Computing* 100:106946. <https://doi.org/10.1016/j.asoc.2020.106946>.
- Li, Q., J. Li, X. Zhang, and B. Zhang. 2021. "A Whale Optimization Algorithm for Distributed Flow Shop with Batch Delivery." *Soft Computing* 25 (21): 13181–13194. <https://doi.org/10.1007/s00500-021-06099-0>.
- Li, W., X. Chen, J. Li, H. Sang, Y. Han, and S. Du. 2022. "An Improved Iterated Greedy Algorithm for Distributed Robotic Flowshop Scheduling with Order Constraints." *Computers & Industrial Engineering* 164:107907. <https://doi.org/10.1016/j.cie.2021.107907>.
- Lohmer, J., and R. Lasch. 2021. "Production Planning and Scheduling in Multi-factory Production Networks: A Systematic Literature Review." *International Journal of Production Research* 59 (7): 2028–2054. <https://doi.org/10.1080/00207543.2020.1797207>.
- Mavrotas, G. 2009. "Effective Implementation of the ϵ -constraint Method in Multi-objective Mathematical Programming Problems." *Applied Mathematics and Computation* 213 (2): 455–465. <https://doi.org/10.1016/j.amc.2009.03.037>.
- Montgomery, D. C. 2017. *Design and Analysis of Experiments*. Arizona State University, Tempe, Arizona: John Wiley & Sons.
- Moon, C., J. Kim, and S. Hur. 2002. "Integrated Process Planning and Scheduling with Minimizing Total Tardiness in Multi-plants Supply Chain." *Computers & Industrial Engineering* 43 (1): 331–349. [https://doi.org/10.1016/S0360-8352\(02\)00078-5](https://doi.org/10.1016/S0360-8352(02)00078-5).
- Naderi, B., and R. Ruiz. 2010. "The Distributed Permutation Flowshop Scheduling Problem." *Computers & Operations Research* 37 (4): 754–768. <https://doi.org/10.1016/j.cor.2009.06.019>.
- Naderi, B., R. Ruiz, and V. Roshanaei. 2023. "Mixed-Integer Programming Vs. Constraint Programming for Shop Scheduling Problems: New Results and Outlook." *INFORMS Journal on Computing* 35 (4): 817–843. <https://doi.org/10.1287/ijoc.2023.1287>.
- Nawaz, M., E. E. Enscore, and I. Ham. 1983. "A Heuristic Algorithm for the M-Machine, N-Job Flow-Shop Sequencing Problem." *Omega* 11 (1): 91–95. [https://doi.org/10.1016/0305-0483\(83\)90088-9](https://doi.org/10.1016/0305-0483(83)90088-9).
- Neufeld, J. S., S. Schulz, and U. Buscher. 2023. "A Systematic Review of Multi-objective Hybrid Flow Shop Scheduling." *European Journal of Operational Research* 309 (1): 1–23. <https://doi.org/10.1016/j.ejor.2022.08.009>.
- Perez-Gonzalez, P., and J. M. Framinan. 2024. "A Review and Classification on Distributed Permutation Flowshop Scheduling Problems." *European Journal of Operational Research* 312 (1): 1–21. <https://doi.org/10.1016/j.ejor.2023.02.001>.
- Ribas, I., R. Companys, and X. Tort-Martorell. 2019. "An Iterated Greedy Algorithm for Solving the Total Tardiness Parallel Blocking Flow Shop Scheduling Problem." *Expert Systems with Applications* 121:347–361. <https://doi.org/10.1016/j.eswa.2018.12.039>.
- Ruiz, R., Q.-K. Pan, and B. Naderi. 2019. "Iterated Greedy Methods for the Distributed Permutation Flowshop Scheduling Problem." *Omega* 83:213–222. <https://doi.org/10.1016/j.omega.2018.03.004>.
- Ruiz, R., and T. Stützle. 2007. "A Simple and Effective Iterated Greedy Algorithm for the Permutation Flowshop Scheduling Problem." *European Journal of Operational Research* 177 (3): 2033–2049. <https://doi.org/10.1016/j.ejor.2005.12.009>.
- Schulz, S., U. Buscher, and L. Shen. 2020. "Multi-objective Hybrid Flow Shop Scheduling with Variable Discrete Production Speed Levels and Time-of-use Energy Prices." *Journal of Business Economics* 90 (9): 1315–1343. <https://doi.org/10.1007/s11573-020-00971-5>.
- Schulz, S., M. Schönheit, and J. S. Neufeld. 2022. "Multi-objective Carbon-Efficient Scheduling in Distributed Permutation Flow Shops under Consideration of Transportation Efforts." *Journal of Cleaner Production* 365:132551. <https://doi.org/10.1016/j.jclepro.2022.132551>.
- Song, H.-B., J. Lin, and Y.-R. Chen. 2025. "An Effective Two-Stage Heuristic for Scheduling the Distributed Assembly Flowshops with Sequence Dependent Setup times." *Computers & Operations Research* 173:106850. <https://doi.org/10.1016/j.cor.2024.106850>.
- Taillard, E. 1993. "Benchmarks for Basic Scheduling Problems." *European Journal of Operational Research* 64 (2): 278–285. [https://doi.org/10.1016/0377-2217\(93\)90182-M](https://doi.org/10.1016/0377-2217(93)90182-M).
- Thoney, K. A., T. J. Hodgson, R. E. King, M. R. Taner, and A. D. Wilson. 2002. "Satisfying Due-Dates in Large Multi-factory Supply Chains." *IIE Transactions* 34 (9): 803–811.
- Wang, S., Y. Feng, X. Li, and G. Zhang. 2024. "Memetic Discrete Differential Evolution with Domain Knowledge for Blocking Scheduling Distributed Flow Shop with Lot-streaming Constraints." *Expert Systems with Applications* 255:124874. <https://doi.org/10.1016/j.eswa.2024.124874>.
- Wang, Y., Y. Wang, Y. Han, K. Gao, J. Li, and Y. Wang. 2025. "Enhancing Distributed Blocking Flowshop Group Scheduling: Theoretical Insight and Application of an Iterated Greedy Algorithm with Idle Time Insertion and Rapid

- Evaluation Mechanisms.” *Expert Systems with Applications* 271:126600. <https://doi.org/10.1016/j.eswa.2025.126600>.
- Wu, X., and A. Che. 2020. “Energy-Efficient No-wait Permutation Flow Shop Scheduling by Adaptive Multi-objective Variable Neighborhood Search.” *Omega* 94:102117. <https://doi.org/10.1016/j.omega.2019.102117>.
- Xiong, F., S. Chen, N. Xiong, and L. Jing. 2025. “Scheduling Distributed Heterogeneous Non-permutation Flowshop to Minimize the Total Weighted Tardiness.” *Expert Systems with Applications* 272:126713. <https://doi.org/10.1016/j.eswa.2025.126713>.
- Xiong, F., L. Jing, C. Xiang, and Y. Han. 2025. “Order Acceptance and Scheduling under Eligibility, Availability, and Budget Constraints in Distributed Heterogeneous Flow Shop Production.” *Expert Systems with Applications* 280:127586. <https://doi.org/10.1016/j.eswa.2025.127586>.
- Xiong, F., J. Shi, L. Jing, and A. Ping. 2025. “Logic-Based Benders Decomposition Methods for the Distributed Permutation Flow Shop Scheduling Problem with Production and Transportation Cost.” *Computers & Operations Research* 179:107044. <https://doi.org/10.1016/j.cor.2025.107044>.
- Xu, T., F. Zhao, J. Zhang, J. Tang, and H. Zhou. 2025. “A Feature Based Neural Network Model for Distributed Flexible Flow Shop Scheduling considering Worker and Transportation Factors.” *Computers & Industrial Engineering* 201:110917. <https://doi.org/10.1016/j.cie.2025.110917>.
- Yu, X., H. Du, D. Zhou, Q. Wang, and G. Lin. 2024. “Bi-objective Carbon-Efficient Distributed Flow-Shop Scheduling with Multistep Electricity Pricing.” *International Journal of Production Research* 62 (18): 6842–6858. <https://doi.org/10.1080/00207543.2024.2312209>.
- Zhang, B. 2025. “Multi-objective Scheduling for Surface Mount Technology Workshop: Automatic Design of Two-Layer Decomposition-Based Approach.” *International Journal of Production Research* 63 (20): 7570–7590. <https://doi.org/10.1080/00207543.2025.2502106>.
- Zhang, C., Y. Han, Y. Wang, J. Li, and K. Gao. 2025. “An Accelerated Discrete Artificial Bee Colony Algorithm under the Makespan Constraint: Solving the Distributed Blocking Flow Shop Scheduling Problem with Balanced Energy Consumption Costs.” *Expert Systems with Applications* 260:125374. <https://doi.org/10.1016/j.eswa.2024.125374>.
- Zhang, Z.-Q., X.-P. Zhu, Y.-X. Xu, B. Qian, B. Yang, and R. Hu. 2025. “Medhea: Multidimensional Estimation of Distribution Based Hyper-heuristic Evolutionary Algorithm for Energy-Efficient Distributed Assembly No-wait Flow-Shop Scheduling Problem.” *Expert Systems with Applications* 271:126526. <https://doi.org/10.1016/j.eswa.2025.126526>.
- Zhao, F., X. Hu, L. Wang, T. Xu, N. Zhu, Jonrinaldi. 2023. “A Reinforcement Learning-Driven Brain Storm Optimisation Algorithm for Multi-objective Energy-Efficient Distributed Assembly No-wait Flow Shop Scheduling Problem.” *International Journal of Production Research* 61 (9): 2854–2872. <https://doi.org/10.1080/00207543.2022.2070786>.
- Zhao, F., H. Zhang, L. Wang, T. Xu, N. Zhu, and J. Jonrinaldi. 2024. “A Multi-objective Discrete Differential Evolution Algorithm for Energy-Efficient Distributed Blocking Flow Shop Scheduling Problem.” *International Journal of Production Research* 62 (12): 4226–4244. <https://doi.org/10.1080/00207543.2023.2254858>.
- Zhong, Q., Y. Yu, Z. Li, L. Sun, Y. Han, N. M. Matsveichuk, and Y. N. Sotskov. 2026. “An Iterative Greedy Algorithm Based on Neighborhood Search for Energy-Efficient Scheduling of Distributed Permutation Flowshop with Sequence-Dependent Setup Time.” *Expert Systems with Applications* 296:129012. <https://doi.org/10.1016/j.eswa.2025.129012>.
- Zitzler, E., and L. Thiele. 1999. “Multiobjective Evolutionary Algorithms: A Comparative Case Study and the Strength Pareto Approach.” *IEEE Transactions on Evolutionary Computation* 3 (4): 257–271. <https://doi.org/10.1109/4235.797969>.
- Zitzler, E., L. Thiele, M. Laumanns, C. Fonseca, and V. da Fonseca. 2003. “Performance Assessment of Multiobjective Optimizers: An Analysis and Review.” *IEEE Transactions on Evolutionary Computation* 7 (2): 117–132. <https://doi.org/10.1109/TEVC.2003.810758>.

Appendix

Table A1. Combined two-way ANOVA results for IGD and HV metrics for small and large instances including F-value and *p*-value.

Source of Variation	IGD Metric				HV Metric			
	Small		Large		Small		Large	
	<i>F</i>	<i>p</i>	<i>F</i>	<i>p</i>	<i>F</i>	<i>p</i>	<i>F</i>	<i>p</i>
Selection method	2.574	0.109	8.215	0.004	0.012	0.913	1.298	0.255
Destruction intensity	3.403	0.034	0.439	0.645	0.005	0.995	0.026	0.975
Interaction	0.323	0.724	0.044	0.957	0.001	0.999	0.015	0.985

Note: Bold values indicate significance at the $p < 0.05$ level.

Table A2. Tukey's HSD post-hoc analysis for significant factors.

Group 1	Group 2	Mean Diff.	95% CI Lower	95% CI Upper	p_{Tukey}	Sig.
<i>Destruction intensity (IGD, Small instances)</i>						
5	10	−0.0115	−0.0284	0.0054	0.2469	No
5	15	−0.0186	− 0.0355	− 0.0017	0.0272	Yes
10	15	−0.0071	−0.0240	0.0098	0.5873	No
<i>Selection method (IGD, Large instances)</i>						
SelectRnd	SelectBest	0.0121	0.0038	0.0203	0.0042	Yes